

UnifOMR: Oblivious Message Retrieval with Near-optimal Concrete Efficiency

Ben Fisch	Zeyu Liu	Eran Tromer	Yunhao Wang
<i>Yale University</i>	<i>Yale University</i>	<i>Boston University</i>	<i>Yale University</i>
benjamin.fisch@yale.edu	zeyu.liu@yale.edu	tromer@bu.edu	yunhao.wang@yale.edu

May 9, 2026

Abstract

End-to-end encryption guarantees message confidentiality but does not hide metadata such as communication patterns among senders and recipients, or their identities. Oblivious Message Retrieval (OMR) is a cryptographic protocol that enables servers to assist recipients in retrieving their messages from a database without learning the mapping between messages and recipients, thereby protecting such metadata.

This paper investigates two central questions of OMR: (1) What is the precise relationship between OMR and the better-studied primitive of Private Information Retrieval (PIR)? (2) Can OMR schemes achieve concrete efficiency comparable to state-of-the-art PIR protocols?

We show that OMR with a property we call *strong detection-key-unlinkability* is at least as hard as PIR, and that existing OMR constructions already satisfy this property. This PIR-to-OMR reduction has low overhead, suggesting that OMR cannot be made substantially more efficient than PIR.

We then present UnifOMR, which achieves $20\times$ to $1080\times$ faster server runtime over the state-of-the-art SophOMR under practical parameter settings. For 2^{19} messages of 612 bytes each, UnifOMR completes in only ~ 25 seconds with 4 MB of communication, compared to > 1250 seconds and 260 KB for SophOMR. These gains come with two trade-offs: an asymptotically linear digest size (albeit with small constants), and two rounds of interaction between the detector and the client.

Furthermore, crucially, UnifOMR uses batch PIR as a black-box component, which in our experiments accounts for 50–92% of the server runtime. Thus, UnifOMR nearly matches the aforementioned lower bound concretely (for databases of 2^{16} to 2^{23} messages, each with 612 to 3060 bytes), given the status quo of batch PIR.

Thus, our results provide both a theoretical foundation for understanding OMR and a practical step toward making it deployable in real-world privacy-preserving systems.

Contents

1	Introduction	4
1.1	Our Contribution	5
1.2	Related Works	6
2	Technical Overview	6
2.1	The Efficiency Bound of OMR	6
2.2	A New OMR Construction	8
3	Oblivious Message Retrieval	10
3.1	Models	10
3.2	Definition	11
4	Preliminary	17
4.1	Notation	17
4.2	Hardness Assumption	17
4.3	Private Information Retrieval	17
4.4	Public Key Encryption	19
4.5	Additively Homomorphic Encryption	21
5	The Relationship Between OMR and PIR	22
5.1	OMR Implies PIR	22
5.2	OMR Implies Key-private PKE	25
6	The Main OMR Construction	27
6.1	Overview of Retrieval based on Additively Homomorphic Encryption	28
6.2	Full Construction with Reduced Communication	29
6.3	Additional Optimizations	33
7	Evaluation	33
7.1	Parameter Selection	33
7.2	Comparison and Benchmark	36
8	Conclusion	38
	Acknowledgements	39
	References	40
A	Additional OMR Definition	48
B	BFV Additive Homomorphic Encryption	48
C	Additively Homomorphic Secret Key Encryption Implies PIR	49

D	Additional Discussion on Evaluation	52
D.1	Baselines	52
D.2	Additional Benchmarks	52
D.2.1	Parameter Set 1 Efficiency Breakdown	52
D.2.2	Parameter Set 2 Efficiency Breakdown	52
D.3	Alternative Batch PIR	52
D.4	Comparison to InstantOMR [54]	54
E	Open Questions	55

1 Introduction

In message delivery systems, end-to-end encryption is a baseline requirement for ensuring privacy. However, this alone is insufficient. In particular, metadata, such as the identities of senders and recipients, the communication graph between them, or even the timestamps of messages alone can leak highly sensitive information [7, 67, 72, 74, 87].

As such, protecting metadata, especially *recipient privacy*, is essential in systems that aim to provide strong privacy guarantees, including private messaging [91, 28, 64, 16, 9] and privacy-preserving blockchains [76, 12, 45, 18]. A straightforward solution is for the recipient to download and attempt to decrypt all messages using their secret key, retaining only the ones successfully decrypted. However, this imposes substantial communication and computation costs, especially for resource-constrained clients such as mobile users. A natural question is whether this task can be securely outsourced to a server, without compromising recipient privacy.

To address this, Liu and Tromer introduced the notion of *Oblivious Message Retrieval (OMR)* in 2022 [58], a primitive designed precisely to enable such outsourcing (and we survey other related primitives in Section 1.2). In an OMR system, *senders* transmit messages to *recipients* without revealing recipient identities. Each message includes a *payload* (e.g., a transaction or photo) and a *clue*, generated using the recipient’s *clue key*. These messages are posted to a public *bulletin board* (e.g., a blockchain ledger or messaging server database). To retrieve their messages, a recipient sends a *detection key* to a server, called a *detector*, which uses it to scan the bulletin board and produce a *digest*. The recipient then decodes the digest to recover the payloads addressed to them. We describe the system model of OMR in more detail in Section 3.1.

Despite several follow-up works that improve various aspects of OMR [59, 60, 47, 57, 51, 27], two important open questions remain. The first concerns the relationship between OMR and the well-studied primitive of Private Information Retrieval (PIR) [26]. At first glance, one might wonder whether PIR is already sufficient to achieve OMR. Indeed, several OMR constructions have considered using PIR as a subroutine [58, 47, 27]. However, OMR introduces unique challenges that are not obviously addressed by PIR. This raises the following fundamental question:

What is the relationship between OMR and PIR? Is OMR weaker or at least as hard as PIR?

This question is not only theoretically interesting for understanding the boundaries between primitives but also practically significant. PIR is known to have tight lower bounds on efficiency [26, 10, 92]. If OMR is at least as powerful as PIR, then these bounds would also apply to OMR. Furthermore, if the reduction is tight (i.e., an explicit construction of PIR from OMR involves little efficiency overhead), then OMR can hardly outperform the best known PIR schemes. Understanding this relationship is thus key to setting expectations for OMR performance.

The second question concerns concrete efficiency. All known single-server OMR protocols rely on fully homomorphic encryption (FHE) [58, 59, 60, 57, 51, 27], and require evaluating deep circuits, which severely limits performance. For example, the state-of-the-art single-server construction SophOMR [51] takes over 1250 seconds to generate a response for a recipient when there are 2^{19} messages in total, each containing a 612-byte payload, with 50 messages pertinent to the recipient. As acknowledged in prior works [58, 60, 51], the detector’s runtime is the main performance bottleneck (see also Section 8). In contrast, batch PIR (e.g., VecBatchPIR [73]) can handle the same parameters in < 15 seconds. This highlights a substantial performance gap between the two primitives. Thus, the second question is:

Can we construct OMR schemes whose server-side performance approaches that of state-of-the-art batch PIR?

In this work, we address both questions. First, we prove that OMR, with an additional property satisfied by all existing single-server constructions, is strictly stronger than PIR as a cryptographic primitive. Moreover, this PIR-to-OMR reduction is tight. Second, we present a new OMR construction, **UnifOMR**, that is $\sim 50\times$ faster than **SophOMR**, and within a factor of 2 of the **PIRANA** scheme in terms of server-side runtime.

1.1 Our Contribution

Efficiency lower bound. We first establish the efficiency (lower) bound of OMR via PIR, both asymptotically and concretely:

- We prove that an OMR construction can be used to build PIR with essentially *no overhead*, as long as the OMR scheme is strongly detection-key-unlinkable, a notion extended from the detection-key-unlinkability property introduced in [58]. While newly defined in this paper, this property is satisfied by all the existing single-server OMR constructions [58, 59, 60, 57, 51], as we will show.

Furthermore, this low-overhead reduction implies that such an OMR construction *cannot* be more concretely efficient than a PIR scheme with essentially the same database size.

- As a side result, we show that OMR is at least as hard as public key encryption with two special properties, key-privacy and wrong-key-decryption (Section 4.4). While this does not directly prove that PIR does not imply OMR, it provides strong evidence towards this direction and strengthens the theoretical understanding of OMR.

A new OMR construction. We then present a new OMR construction, **UnifOMR** available at [77]), unifying the concrete performance of OMR and PIR as follows:

- **UnifOMR** relies only on additively homomorphic encryption (AHE) instead of FHE as in prior works. Specifically, we use RLWE-based AHE together with batch PIR to build an OMR construction.
- We implement **UnifOMR** and show that with $N = 2^{19}$ messages each with 612 bytes (parameters in prior works [58, 59, 60, 57, 51]), our runtime is ≈ 25 seconds, which is $> 50\times$ faster than the state-of-the-art OMR construction, **SophOMR** [51]. Furthermore, our detection key size is only 31MB, about $\sim 5\times$ smaller than the **SophOMR**. This is achieved at the cost of having the digest size being $\sim 4\text{MB}$, instead of $\sim 200\text{KB}$ as in **SophOMR**.

Furthermore, for other parameters (e.g., a larger number of pertinent messages), we achieve a similar or better runtime improvement, even with digest size overhead $< 2\times$.

These are achieved at the cost of having two rounds of communication between the detector and the recipient, and having a digest size asymptotically linear in N (thereby also recipient runtime), despite being concretely small.

- Lastly, based on our benchmarks, for **UnifOMR**, the batchPIR component takes 50-92% of the total detector runtime (dependent on parameters). This means that **UnifOMR** is very close to the (concrete) runtime lower bound by the reduction above, given the status quo of batchPIR¹.

¹Specifically, our performance profiling is based on the current state-of-the-art batchPIR. If a more efficient batchPIR exists, the relative costs would change and the cost of batchPIR may no longer dominate.

1.2 Related Works

Oblivious Message Retrieval. OMR [58] introduces a message retrieval primitive that achieves full recipient privacy. GOMR [59] extends this model to a group setting and proposes an optimized OMR construction. Both constructions rely on a hybrid approach combining the PVW encryption scheme with a leveled BFV homomorphic encryption scheme. PerfOMR [60] improves upon these by replacing the PVW encryption with a tailored RLWE-based scheme and modifying the underlying BFV-based retrieval circuit. This results in improved efficiency over [58, 59] in terms of detector runtime, clue key size, and clue size. SophOMR [51] introduces additional optimizations that further reduce the detector runtime, detection key size, and digest size. It currently represents the state-of-the-art among *single-server* OMR constructions on a single-thread CPU. A recent work InstantOMR [54] introduces low-latency OMR which outperforms SophOMR with large number of cores or small amount of messages. Another recent work [17] introduces ways to accelerate OMR using GPU.

A recent line of work explores multi-server OMR. Homerun [47] constructs a two-server OMR protocol that is concretely more efficient than existing single-server schemes. However, its security model is restrictive: it requires two non-colluding servers and also all the servers and clients to be semi-honest (or allow malicious clients but require the servers to be “almost” honest: i.e., cannot impersonate a malicious client). A follow-up work [27] strengthens the security model by allowing fully malicious but non-colluding servers (and malicious clients). While the security is improved, the performance becomes comparable to SophOMR (within a factor of 2). Since our focus is on single-server OMR, we do not compare extensively with this line of work.

Fuzzy Message Detection. Fuzzy Message Detection (FMD) [9, 80] focuses on decoy-based security. While this primitive admits highly efficient constructions, the security guarantees are relatively weak, as discussed in [85]. Therefore, we do not compare directly with these constructions.

Private Signaling. Like OMR, Private Signaling (PS) [66, 46] aims to provide full recipient privacy. However, prior PS constructions rely on Trusted Execution Environments (TEEs), which constitute a strong assumption. Many works have shown that existing TEEs are vulnerable to side-channel attacks that can leak secrets [89]. Although [46] achieves excellent scalability (with runtime growing only polylogarithmically in the number of messages), we do not compare directly with it, as we assume a standard software model.

Another construction is proposed in [66], assuming the same assumption as Homerun [47]. Moreover, its performance is concretely slower than the single-server OMR constructions. Thus, we do not include it in our comparisons either.

2 Technical Overview

We now give an overview of our techniques. The overview assumes familiarity with the model and definition of OMR (both presented in Section 3), PIR, and PKE (both recalled in Section 4).

2.1 The Efficiency Bound of OMR

This section summarizes how we establish the efficiency (lower) bound of OMR by showing that OMR is a strictly stronger primitive than PIR, and building PIR from OMR has essentially no overhead.

Why PIR and OMR differ. Before presenting a more formal argument, we first clarify why OMR is intuitively “harder” than PIR.

On the one hand, both OMR and PIR allow a recipient or client to retrieve data entries from a database without revealing which entries are being accessed, making them very similar at first glance. On the other hand, the key distinction is that in PIR, the client *knows* the indices of the entries it wishes to retrieve. In contrast, in OMR, the recipient does *not* know the positions of its messages in the database. This challenge cannot be addressed by keyword PIR either (where retrieval is based on keywords rather than indices): In OMR, the recipient cannot rely on a keyword known to both the recipient and the sender, since no secret is shared between them in the OMR model. If such a keyword exists, the adversary also knows it and thus breaks privacy.

Instead, OMR requires each message to include an encrypted tag (the “clue”) which identifies the intended recipient. Only the recipient with an appropriate secret key can recognize and decrypt it. For all other parties, the clue appears to be a uniformly random string. This mechanism fundamentally differs from PIR, where the index or keyword is fixed and known to the client. While the clue in OMR can serve as a stand-in for the index in PIR (intuitively enabling a reduction from PIR to OMR), the reverse is intuitively not true: an index or keyword cannot substitute for the random-looking clue required in OMR.

In the rest of this section, we give more formal arguments extending from this intuition.

OMR implies PIR. First, we show that OMR with strong-key-unlinkability (which means that two freshly generated detection keys generated by *the same* recipient are indistinguishable from two detection keys generated by *two distinct* recipients, even given the two recipients’ secret clue keys) implies PIR with essentially no overhead. In other words, an OMR construction can be used to build an efficient PIR construction with almost no efficiency loss (in terms of server computation and communication). We informally capture this as follows and sketch our proof intuition below. It is formally stated and proven in Theorem 5.1.

Theorem 2.1 (Informal). *An OMR construction can be used to construct PIR with the same number of items in the database with essentially the same performance.*

At a high level, given an OMR construction OMR, we build a PIR scheme as follows. For a database of N entries, let each entry $\text{DB}[i]$ be associated with a clue c_i generated using some $\text{pk}_{\text{clue}_i}$ with the corresponding $\text{sk}_{\text{clue}_i}$. To retrieve entry i , the client uses $\text{sk}_{\text{clue}_i}$ to generate a detection key pair $(\text{sk}_{\text{det}}, \text{pk}_{\text{det}})$. This pk_{det} is then sent to the server, which also holds DB (containing all the N clues above), to generate a digest using OMR. By the completeness of OMR, the client can decode the digest to obtain $\text{DB}[i]$.

Security-wise, by OMR’s strong key-unlinkability, even if the server knows $\text{sk}_{\text{clue}_i}$, pk_{det_i} does not leak any information about i . Therefore, the server does not learn anything about i , satisfying the privacy requirement of PIR.

Efficiency-wise, the PIR server runtime is essentially identical to the OMR detector runtime, and the communication cost between the PIR server and client is the detection key size plus the digest size.

Conceptually, this finishes the reduction from PIR to OMR. There are still some gaps left, such as when the clues are generated and how the false negative rate or false positive rate affects this reduction. These are more formally addressed in the proof of Theorem 5.1. We also discuss an alternative reduction from PIR to OMR that achieves slightly different efficiency trade-offs, but this also follows a similar high-level intuition. Thus, we conclude that OMR implies PIR.

OMR implies key-private PKE. We additionally show that OMR implies PKE with two special properties, key-privacy and wrong-key-decryption. Key-privacy (Definition 4.5) means that ciphertexts under two different public keys are indistinguishable [11]. Wrong-key decryption (Definition 4.5) means that decrypting under a different but honestly generated key results in random plaintext [58].

This reduction does not directly prove that OMR is strictly stronger than PIR, since non-trivial single-server PIR also implies PKE [31, 38]. Nonetheless, it provides evidence that OMR is stronger: If PIR implies OMR, additively homomorphic symmetric key encryption (AHSKE, which implies PIR as discussed in Appendix C) implies key-private PKE. To our knowledge, this is still open and unlikely (Remark C.3).

Note that this result is only of independent interest, orthogonal to the main contribution of this work: building an efficient OMR scheme, concretely close to the efficiency lower bound.

Theorem 2.2 (Informal). *OMR implies PKE with key-privacy and wrong-key-decryption properties.*

Indeed, given an OMR construction, we can construct a PKE scheme as follows. The recipient generates two clue key pairs $(\text{sk}_{\text{clue0}}, \text{pk}_{\text{clue0}})$ and $(\text{sk}_{\text{clue1}}, \text{pk}_{\text{clue1}})$, and the PKE public key is $\text{PKE.pk} := (\text{pk}_{\text{clue0}}, \text{pk}_{\text{clue1}})$. To encrypt a bit $b \in \{0,1\}$ under this public key, the sender uses $\text{pk}_{\text{clue}b}$ to generate a clue c_b , with some dummy random payload $r \leftarrow_{\$} \{0,1\}^\lambda$, and sends $\text{ct} = (c_b, r)$ as the ciphertext.

To decrypt, the recipient uses the detection key $\text{pk}_{\text{det}b}$ (generated from $\text{sk}_{\text{clue}b}$) to perform retrieval on the OMR board that consists of the single message ct , and then decodes the generated digest into M_b using $\text{sk}_{\text{det}b}$, for $b \in \{0,1\}$. If the encrypted bit is 0, M_0 contains r by the completeness of OMR. If the encrypted bit is 1, M_0 *does not* contain r by the soundness of OMR, while M_1 contains r by the completeness of OMR. This guarantees the correctness of the PKE scheme.

CPA security holds by the privacy of OMR: no PTT algorithm is able to distinguish a clue generated via pk_{clue0} or pk_{clue1} unless holding sk_{clue0} or sk_{clue1} . Similarly, key-privacy is guaranteed. Since wrong-key-decryption requires another minor change to the construction, we defer it to Section 5.2.

This then concludes the intuition behind why OMR implies key-private PKE.

2.2 A New OMR Construction

Now, we discuss our new OMR construction UnifOMR. The main workflow of our UnifOMR is summarized in Fig. 1. Step 5 (Steps 5.1-3) is the retrieval process, which is the main inefficiency of all prior single-server OMR constructions and the main new technical contribution of UnifOMR.

Identify the source of inefficiency. Observe that the main computational inefficiency of existing single-server OMR constructions is mainly incurred by the use of (leveled) fully homomorphic encryption schemes: The circuits they compute are very deep, usually > 20 levels of multiplication.

Such a deep circuit is used to achieve the so-called *compactness*. In essence, it means that the digest size should be independent of the bulletin board size N (at most polylogarithmically dependent). This is asymptotically attractive, since it means that the data that the recipient needs to download is almost independent of N . However, our main insight is that, while compactness is theoretically interesting, achieving it is unnecessarily costly in practice. Concretely, for parameters used in prior works [58, 59, 60, 51], for 2^{19} messages each with 612 bytes, the digest size is achieved

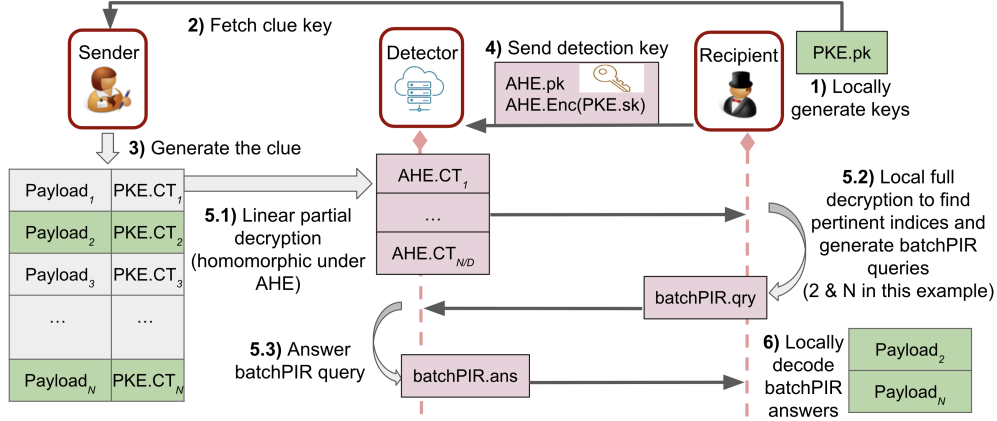


Figure 1: Workflow of UnifOMR.

to be $\sim 260\text{KB}$, while the detector runtime is > 1250 seconds. On the other hand, we observe that if we relax this asymptotic requirement, we still get a construction that is reasonably small in terms of concrete digest size (only $\sim 4\text{MB}$), and this gap from SophOMR’s digest size can be further reduced given a larger payload size (see Section 7). Simultaneously, the detector runtime is much smaller, since we replace the deep circuit with a simple linear circuit, introduced below.

Client-aided retrieval. Recall that, as in prior works, to send a message to a recipient with a key pair (sk_{clue}, pk_{clue}) , the sender encrypts a vector of zeros 0^ℓ under pk_{clue} as a clue c . Then, to see whether a message is pertinent or not, the recipient decrypts c using sk_{clue} . If the message is pertinent, the decryption results in 0^ℓ . Otherwise, the decryption results in a random binary vector, which is not 0^ℓ with $1 - \text{negl}(\ell)$ probability. This property is achieved by the RLWE-based PKE scheme we use (see Definition 4.6 for details).

To outsource this process to the detector, one could encrypt sk_{clue} under FHE and let the detector homomorphically perform this decryption on behalf of the recipient, as in prior works [58, 59, 60, 57, 51]. As aforementioned, this results in a deep homomorphic circuit. Observe that under RLWE-based PKE schemes, the decryption circuit has two components: a linear operation and a range check, and the latter is the main cause of the deep circuit. Thus, we only ask the detector to perform the linear operation component under FHE (even an additively homomorphic encryption, AHE, is sufficient).

Then, the detector sends this “partially decrypted result” back to the recipient. The recipient decrypts the FHE ciphertexts, performs a range check over the plaintexts (i.e., finishing the clue ciphertext decryption), and then finds out which messages are pertinent. Lastly, the recipient uses a batch Private Information Retrieval (batchPIR) with these pertinent indices to obtain the corresponding payloads. This completes the high-level idea of UnifOMR.

Advantages over the strawman solution. One may observe that there is a strawman solution where the recipient simply downloads all the clues and locally decrypts all of them. Then, the recipient performs a batchPIR as in UnifOMR. Compared to UnifOMR, this solution is much less efficient for the recipients. Specifically, the recipient needs to perform N PKE ciphertext decryptions, and the communication cost (ignoring the batchPIR component) is $|c| \cdot N$ for clue size $|c|$ and N messages. Both can be a huge burden for a light client. In comparison, UnifOMR leverages the SIMD property of the underlying AHE scheme to pack the results via partial decryption, which then

only requires N/D AHE instead of N PKE ciphertexts to be sent, for some packing parameter D . Similarly, the recipient only needs to decrypt N/D AHE ciphertexts instead of N PKE ciphertexts. Concretely, this saves 1-3 orders of magnitude for communication and computation (detailed in Section 7).

Further reducing the communication. To concretely realize the idea, UnifOMR uses RLWE for both PKE (for clue generation) and AHE (for evaluating partial decryption homomorphically), both of which incur “errors”. We observe that one could potentially merge the two error terms for the final digest and then apply “modulus switching” procedure to further reduce the communication cost. However, this may not be naively achieved if the partial decryptions are done inside the “slots” of the AHE scheme (Section 4.5). Thus, instead, we use coefficient encoding for the clues and encode sk_{clue} as constant polynomials to make the merging possible. We present the details of the communication reduction technique in Section 6.2 and present some additional optimizations in Section 6.3.

As mentioned, our resulting construction UnifOMR takes only about 25 seconds to retrieve 2^{19} messages, > 50 times faster than the state-of-the-art OMR construction SophOMR [51]. This advantage can be as high as $1080\times$ for the parameters we tested (see Section 7). Furthermore, our detection key size is only 31 MB, about 5x smaller.

3 Oblivious Message Retrieval

This section introduces the models (threat model and system model) of Oblivious Message Retrieval (OMR) and its formal definition.

3.1 Models

Threat model. For completeness and soundness, OMR assumes honest-but-curious adversaries. The adversary may collude with all parties in the system, including detectors, senders, and recipients, except for the protection target. They are allowed to view all the public information, generate new messages honestly, and generate keys honestly (which can be used to generate messages). For privacy, we allow all parties in the system to be *malicious* and colluding, except for the protection target. In this case, the adversary may behave *arbitrarily*. Note that in both cases, the adversary is computationally-bounded (i.e., polynomial time).

In [58], the authors have introduced the DoS-resistance model, where the senders and the recipient may be malicious while still guaranteeing completeness and soundness. In [57], the authors proposed a general way to convert a non-DoS-resistant lattice-based OMR to a DoS-resistant, with very little overhead (about 20%). This conversion also applies to our scheme. Thus, similar to prior works [60, 51], we focus on the standard model, which is sufficient to achieve the DoS-resistant OMR with little overhead using [57].

Detailed system model. We start by recalling the model and the problem of Oblivious Message Retrieval. Note that this is taken directly from [58]. The system components and their high-level properties are as follows.

A *bulletin board* (or *board* for short), denoted BB, contains N messages, e.g., blockchain transactions or texts. Each message is sent from some sender and addressed to some recipient, whose identities are supposed to remain private.

A message consists of a pair (x_i, c_i) where x_i is the message *payload* to convey, and c_i is a *clue* string which helps notify the intended recipient (and only them) that the message is addressed to them. The clue is generated using the recipient's *public clue key* pk_{clue} (corresponding to a secret clue key sk_{clue}).

We denote the payload space $\mathcal{P}$, and the clue space $\mathcal{C}$ (typically a ciphertext of some PKE scheme). The whole board BB (i.e., all payloads and clues) is public. (In applications, the payloads will typically be end-to-end encrypted.)

At any time, any potential recipient p may retrieve the messages addressed to them in BB . We call these messages *pertinent* (to p), and the rest are *impertinent*.

A server, called a *detector*, helps the recipient p retrieve the pertinent messages' payloads in a privacy-preserving way. The recipient gives the detector their *public detection key* pk_{det} (corresponding to a secret detection key sk_{det}), and a bound $\bar{k}$ on the number of pertinent messages they expect to receive. The detector then utilizes all of the messages (including both clues and payloads) in BB to generate string dig , called the *digest*, and sends it to the recipient p . The digest dig should be much smaller than the board BB (ideally, proportional to $\bar{k}$).

The recipient p processes dig to recover all of the pertinent messages with high probability, assuming a semi-honest detector (for correctness, but malicious for privacy) and that the number of pertinent messages did not exceed $\bar{k}$. The *false negative rate* (probability that a pertinent message is not recovered from the digest) is denoted by ϵ_n . The *false positive rate* (probability that an impertinent message is output by the recovery procedure) is denoted by ϵ_p . Both ϵ_n and ϵ_p are small (e.g., under 2^{-30} and 2^{-20} respectively).

There may be many detectors, each supporting many recipients.

3.2 Definition

We first recall the definition of Oblivious Message Retrieval (with some minor changes explained below). Then, we extend/strengthen one of the properties proposed by [58] and show that all existing single-server OMR works satisfy this extended property.

Definition 3.1 (*w*-round Oblivious Message Retrieval (OMR)). A *w*-round Oblivious Message Retrieval scheme has the following PPT algorithms:

- $\text{pp} \leftarrow \text{GenParam}(1^\lambda, \epsilon_p, \epsilon_n)$: takes a security parameter λ , a false positive rate ϵ_p , a false negative rate ϵ_n , and outputs a public parameter pp .
- $(\text{sk}_{\text{clue}}, \text{pk}_{\text{clue}}) \leftarrow \text{GenClueKey}(\text{pp})$: takes public parameter pp ; outputs a clue secret key sk_{clue} and public key pk_{clue} .
- $(\text{sk}_{\text{det}}, \text{pk}_{\text{det}}) \leftarrow \text{GenDetKey}(\text{pp}, \text{sk}_{\text{clue}})$: takes public parameter pp and a secret clue key sk_{clue} ; outputs a detection secret key sk_{det} and public key pk_{det} .
- $c \leftarrow \text{GenClue}(\text{pp}, \text{pk}_{\text{clue}})$: takes the public parameter pp , a clue key pk_{clue} ; outputs a clue $c \in \mathcal{C}$.
- Retrieval is a *w*-round protocol between the detector and the recipient for $w \geq 1$, and then for round $i \in [w]$:

- $\text{dig}_i \leftarrow \text{Retrieve}_i(\text{pp}, \text{BB}, \text{pk}_{\text{det}}, \bar{k}, \text{aux}_i)$: the algorithm takes the public parameter pp , a bulletin board $\text{BB} = \{(x_1, c_1), \dots, (x_N, c_N)\}$ of size N , a detection key pk_{det} , an upper bound $\bar{k}$ on the number of pertinent messages addressed to that recipient, and some auxiliary input aux_i (which is $\perp$ for $i = 0$, i.e., the first round); outputs a digest dig_i for round i .
- $\text{stR}_{i+1}, \text{aux}_{i+1} \leftarrow \text{Decode}_i(\text{pp}, \text{sk}_{\text{det}}, \bar{k}, \text{dig}_i, \text{stR}_i)$: takes the public parameter pp , the bound $\bar{k}$, the detection secret key sk_{det} , the digest dig_i , and (secret) recipient state stR_i (which is $\perp$ for $i = 0$); outputs a recipient state stR_{i+1} and auxiliary information aux_{i+1} for the next round.

We first define board generation for completeness and soundness below:

Definition 3.2 (Board Generation). Given pp , and the size of bulletin board N : arbitrarily choose the number of recipients $1 \leq p \leq N$, and a partition of set $[N]$ into p subsets $S_1, \dots, S_p$ representing the indices of messages addressed to each party. For each recipient $i \in [p]$: generate keys $(\text{sk}_{\text{clue}_i}, \text{pk}_{\text{clue}_i}) \leftarrow \text{GenClueKey}(\text{pp})$, $(\text{sk}_{\text{det}_i}, \text{pk}_{\text{det}_i}) \leftarrow \text{GenDetKey}(\text{pp}, \text{sk}_{\text{clue}_i})$, and for each $j \in S_i$, generate $c_j \leftarrow \text{GenClue}(\text{pk}_{\text{clue}_i}, x_j)$. Lastly, arbitrarily choose unique payloads $(x_1, \dots, x_N) \in \mathcal{P}^N$ (for payload space $\mathcal{P}$). Then, output the board $\text{BB} = \{(x_1, c_1), \dots, (x_N, c_N)\}$, the set S_1 , and $(\text{sk}_1 = (\text{sk}_{\text{clue}_1}, \text{sk}_{\text{det}_1}), \text{pk}_1 = (\text{pk}_{\text{clue}_1}, \text{pk}_{\text{det}_1}))$.²

The scheme must satisfy the following properties:

- (Completeness) For any $\lambda > 0, 0 < \epsilon_p, \epsilon_n < 1$: Let $\text{pp} \leftarrow \text{GenParam}(1^\lambda, \epsilon_p, \epsilon_n)$. Set any $N = \text{poly}(\lambda)$, and $0 < \bar{k} \leq N$. Let a board BB , a set S of pertinent messages, and a key pair $(\text{sk}, \text{pk} = (\text{pk}_{\text{clue}}, \text{pk}_{\text{det}}))$ be generated as in Definition 3.2 for any choice of p , partition and payloads therein. For $i \in [k]$, let $\text{dig}_i \leftarrow \text{Retrieve}(\text{BB}, \text{pk}_{\text{det}}, \bar{k}, \text{aux}_i)$ and $\text{stR}_{i+1}, \text{aux}_{i+1} \leftarrow \text{Decode}_i(\text{pp}, \text{sk}_{\text{det}}, \bar{k}, \text{dig}_i, \text{stR}_i)$, where $\text{aux}_0, \text{stR}_0 = \perp$. Let $k = |S|$ (the number of pertinent messages in S). Then either $k > \bar{k}$ and $\text{stR}_w = \text{overflow}$, or:

$$\Pr[x_j \in \text{stR}_w \mid j \in S] \geq (1 - \epsilon_n - \text{negl}(\lambda)), \text{ for all } j \in [N] .$$

- (Soundness) For the same quantifiers as in Completeness:

$$\Pr[x_j \in \text{stR}_w \mid j \notin S] \leq (\epsilon_p + \text{negl}(\lambda)), \text{ for all } j \in [N] .$$

- (Computational Privacy) For any $\lambda > 0, 0 < \epsilon_p, \epsilon_n < 1$, for any PPT adversary $\mathcal{A}$, and $P, N = \text{poly}(\lambda)$: It holds that

$$\left| \Pr[\text{OMR-SEC}_{\text{OMR}, P, \epsilon_n, \epsilon_p, N}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq 1/2 + \text{negl}(\lambda) .$$

where OMR-SEC is defined in Fig. 2.

Definition changes from [58]. We make three minor changes to the interfaces:

²That is, S_1 is the indices of messages pertinent to the recipient whose keys are sk_1, pk_1 , which WLOG is the first recipient.

$\text{OMR-SEC}_{\text{OMR}, P, \epsilon_n, \epsilon_p, N}^{\mathcal{A}}(1^\lambda)$ <pre> 1 : $\text{pp} \leftarrow \text{GenParams}(1^\lambda, \epsilon_p, \epsilon_n)$ 2 : for $i \in [2]$: 3 : $(\text{sk}_{\text{clue}_i}, \text{pk}_{\text{clue}_i}) \leftarrow \text{GenClueKey}(\text{pp})$ 4 : $(\text{sk}_{\text{det}_i}, \text{pk}_{\text{det}_i}) \leftarrow \text{GenDetKey}(\text{pp}, \text{sk}_{\text{clue}_i})$ 5 : $b \xleftarrow{\\$} \{0, 1\}, c \leftarrow \text{GenClue}(\text{pp}, \text{pk}_{\text{clue}_b})$ 6 : $\text{st} \leftarrow (\text{pp}, (\text{pk}_{\text{clue}_i}, \text{pk}_{\text{det}_i})_{i \in [2]}, P, \epsilon_n, \epsilon_p, N, c)$ 7 : for $j \in [P]$: 8 : $(b_j, \text{st}, \bar{k}_j \in [N]) \leftarrow \mathcal{A}(\text{st}, j)$ 9 : $\text{stR}_{j,0}, \text{aux}_{j,0} \leftarrow \perp$ 10 : for $i \in [w]$: 11 : $\text{dig}_{j,i}, \text{st} \leftarrow \mathcal{A}(\text{aux}_{j,i}, \text{st})$ 12 : $\text{stR}_{j,i+1}, \text{aux}_{j,i+1} \leftarrow$ 13 : $\text{Decode}_i(\text{pp}, \text{sk}_{\text{det}_{b_j}}, \bar{k}_j, \text{dig}_{j,i}, \text{stR}_{j,i})$ 14 : $b' \leftarrow \mathcal{A}(\text{st})$ 15 : return $b = b'$ </pre>
--

Figure 2: OMR-SEC: security game for OMR privacy.

- We divide KeyGen to be GenClueKey and GenDetKey. This is to ease the definition of key-unlikability introduced in [58] and extension below. Note that [58, Def 9.2] made a similar patch to their definition by defining RegenDetectKey(to define key-unlinkability), and we merged this patch into the main definition.
- The GenClue interface in [58] takes a payload $x \in \mathcal{P}$ as an input, while we remove it. We believe that GenClue takes x as an input is rather counter-intuitive and more restrictive in practice: Specifically, senders need to decide their payloads *first* before generating the clues. Furthermore, to our knowledge, none of the existing OMR constructions uses the payload x during the clue generation.
- Lastly, the most major change: we extend Retrieve and Decode to be an interactive process between the detector and the recipient for w rounds. Specifically, we define the OMR construction to be a w -round OMR construction. For each round $i \in [w]$, the detector runs Retrieve_i (with pk_{det} , $\bar{k}$, and aux_i defined below) and sends back dig_i to the recipient; the recipient, with a secret state stR , uses Decode_i with input dig_i (along with sk_{det} and $\bar{k}$) to update its secret state from stR_i to stR_{i+1} and also to output some additional auxiliary information aux_i for the detector to process in the next round (and $\text{aux}_0 = \perp$). In the last round, the recipient generates stR_w , which contains the necessary information (i.e., all the pertinent messages). For $w = 1$, the definition is the same as the original OMR definition.

To accommodate this change, we change the correctness, soundness, and computational privacy definition as well. The first two are straightforward, while the computational privacy is more complicated. Specifically, we want to make sure a clue c generated via two different

public keys pk_{clue0} and pk_{clue1} is indistinguishable. This needs to be guaranteed even if the adversary (1) sees all the public keys (i.e., line 6 in Fig. 2), and (2) the adversary arbitrarily interacts with the recipient who holds the secret detection key via the retrieval protocol (lines 7-13). For the latter, in more detail, the adversary specifies the secret detection key it wants to interact with (line 8), and acts as if it were the detector (line 8 and 13) to generate the digest dig_j for round $j \in [w]$. Then, the recipient uses the Decode_i to process dig_j . Note that its secret state stR_j is not observable to the adversary (and therefore the very last round of Decode_w is not necessary since nothing can be seen by the adversary). Again, when $w = 1$, this definition is equivalent to the original OMR definition trivially.

This is to allow our construction to take advantage of this flexibility to greatly improve the concrete efficiency. This was already implicitly allowed by [58] since they discussed the possibility of this procedure being interactive, but it was not formally captured in the original definition.

It is easy to see that the resulting definition is stronger (i.e., any OMR construction satisfying the old definition in [58] also satisfies this new definition). Furthermore, the only non-syntactic change is the change to GenClue when $w = 1$, which means that the only situation where a construction satisfying the original OMR definition but not satisfying our definition is when the construction needs to use the payload in a non-trivial way during clue generation, which again could be restrictive in real-world applications.

Compactness. We omit the formal definition of compactness, since we do not use it for our construction. However, for completeness of this paper, we include it in Appendix A.

Definition 3.3. An OMR scheme is *detection-key-unlinkable* if for any $\lambda > 0, 0 < \epsilon_p, \epsilon_n < 1$, for any PPT adversary $\mathcal{A}$, and $P, N = \text{poly}(\lambda)$, it holds that:

$$\left| \Pr[\text{OMR-SDKU}_{\text{OMR}, P, \epsilon_n, \epsilon_p, N}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq 1/2 + \text{negl}(\lambda) .$$

where OMR-DKU is defined in Fig. 3.

Its difference from strong-detection-key-unlinkability (Fig. 4) is shown in blue (i.e., line 6 does not take sk_{clue}).

Remark 3.4. In [58, Section 9], the authors additionally defined the full-key-unlinkability. Essentially, it says that a recipient arbitrarily regenerates public detection keys and public clue keys while the fresh keys cannot be associated with the previously generated keys (given all the public information). This is irrelevant to the gist of this work, so we omit the details. However, we note that UnifOMR indeed possesses this property in a straightforward way.

Strong detection-key-unlinkability. In this work, we define a stronger detection-key-unlinkability property: The adversary is given the randomness used to generate $\text{sk}_{\text{clue}}, \text{pk}_{\text{clue}}$, but still cannot link a public detection key to a specific recipient. Formally, it is defined as follows:

Definition 3.5. A w -round OMR scheme is *strong detection-key-unlinkable* if for any $\lambda > 0, 0 < \epsilon_p, \epsilon_n < 1$, for any PPT adversary $\mathcal{A}$, and $P, N = \text{poly}(\lambda)$, it holds that:

$$\left| \Pr[\text{OMR-SDKU}_{\text{OMR}, N, P, \epsilon_n, \epsilon_p, N}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq 1/2 + \text{negl}(\lambda) .$$

where OMR-SDKU is defined in Fig. 4.

OMR-DKU ^A _{OMR, P, ε_n, ε_p, N} (1 ^λ)	
1 :	pp ← GenParams(1 ^λ , ε _p , ε _n)
2 :	for i ∈ [2] :
3 :	(sk _{clue_i} , pk _{clue_i}) ← GenClueKey(pp)
4 :	(sk _{det_i} , pk _{det_i}) ← GenDetKey(pp, sk _{clue_i})
5 :	b $\xleftarrow{\$}$ {0, 1}
6 :	st ← (pp, (pk _{clue_i}) _{i ∈ [2]} , pk _{det_b} , P, N)
7 :	for j ∈ [P] :
8 :	(k̄ _j ∈ [N], st) ← A ^{GenDetKey(pp, sk_{clue_b})} (st, j)
9 :	stR _{j,0} , aux _{j,0} ← ⊥
10 :	for i ∈ [w] :
11 :	dig _{j,i} , st ← A ^{GenDetKey(pp, sk_{clue_b})} (aux _{j,i} , st)
12 :	stR _{j,i+1} , aux _{j,i+1} ←
13 :	Decode _i (pp, sk _{det_b} , k̄ _j , dig _{j,i} , stR _{j,i})
14 :	b' ← A ^{GenDetKey(pp, sk_{clue_b})} (st)
15 :	return b = b'

Figure 3: OMR-DKU: security game for OMR detection-key-unlinkability.

While this property seems stronger than the standard detection-key-unlinkability property, interestingly, all the existing single-server OMR constructions satisfy such a property (except for the feasibility result in [58, Alg 5], which only served as a stepping stone towards the main constructions; see also Remark 3.7). Specifically, we formalize the following lemma :

Lemma 3.6. OMRp1, OMRp2 [58, Alg 7&8], OMR-OPT [59, Alg 2], PerfOMR1, PerfOMR2 [60, Alg 4& 5], SophOMR [51, Sec 4], adapted to our interfaces (Definition 3.1) as in the proof, are all strong detection-key-unlinkable.

Proof. The proof is straightforward: First, all these constructions use a PKE PKE key pair as the clue key pair and a (leveled) FHE FHE key pair as the detection key pair. Specifically, sk_{clue} := PKE.sk, pk_{clue} := PKE.pk, sk_{det} := FHE.sk, pk_{det} := (FHE.pk, FHE.Enc(FHE.sk, PKE.sk)) (where the last term is PKE.sk encrypted under FHE.sk). Note that the FHE key pair (FHE.sk, FHE.pk) and the PKE key pair (PKE.sk, PKE.pk) are generated fully independently.

Then, adapting to our interfaces is simple:

- For the first change (dividing KeyGen to GenClueKey and GenDetKey): let GenClueKey output sk_{clue} = PKE.sk, pk_{clue} = PKE.pk, and let GenDetKey output sk_{det} := FHE.sk and pk_{det} := (FHE.pk, FHE.Enc(FHE.sk, PKE.sk)).
- For the second change (GenClue does not take the payload as input anymore), since these constructions do not use the input payload at all in their GenClue, this change does not affect anything.

OMR-SDKU _{OMR,P,\epsilon_n,\epsilon_p,N} ^A (1 ^{\lambda})	
1 :	$\text{pp} \leftarrow \text{GenParams}(1^\lambda, \epsilon_p, \epsilon_n)$
2 :	for $i \in [2]$:
3 :	$(\text{sk}_{\text{clue}_i}, \text{pk}_{\text{clue}_i}) \leftarrow \text{GenClueKey}(\text{pp}; r_i)$
4 :	$(\text{sk}_{\text{det}_i}, \text{pk}_{\text{det}_i}) \leftarrow \text{GenDetKey}(\text{pp}, \text{sk}_{\text{clue}_i})$
5 :	$b \xleftarrow{\$} \{0, 1\}$
6 :	$\text{st} \leftarrow (\text{pp}, (\textcolor{blue}{r}_i)_{i \in [2]}, \text{pk}_{\text{det}_b}, P, N)$
7 :	for $j \in [P]$:
8 :	$\text{stR}_{j,0} \leftarrow \perp, \text{aux}_{j,0} \leftarrow \perp$
9 :	$(\bar{k}_j \in [N]) \leftarrow \mathcal{A}(\text{st})$
10 :	for $i \in [w]$:
11 :	$\text{dig}_{j,i}, \text{st} \leftarrow \mathcal{A}(\text{aux}_{j,i}, \text{st})$
12 :	$\text{stR}_{j,i+1}, \text{aux}_{j,i+1} \leftarrow$
13 :	$\text{Decode}_i(\text{pp}, \text{sk}_{\text{det}_b}, \bar{k}_j, \text{dig}_{j,i}, \text{stR}_{j,i})$
14 :	$b' \leftarrow \mathcal{A}(\text{st})$
15 :	return $b = b'$

Figure 4: OMR-SDKU: security game for OMR strong-detection-key-unlinkability. Difference from Fig. 3 highlighted in blue.

- For the last change, their Retrieve and Decode processes are one-round, so these constructions trivially fit in our definition.

It is easy to see that these do not change anything other than the syntax/presentation of all these constructions.

Now, we discuss why these constructions are strong-detection-key-unlinkable: note that FHE.pk is freshly sampled independent from PKE.sk , which means that FHE.pk has the same distribution even given the access to the randomness used to generate $\text{sk}_{\text{clue}} = \text{PKE.sk}$, $\text{pk}_{\text{clue}} = \text{PKE.pk}$. Then, by the CPA security of FHE, it holds that $\text{FHE.Enc}(\text{FHE.sk}, \text{PKE.sk})$ is (computationally) the same distribution given such access as well. Therefore, $\text{pk}_{\text{det}} := (\text{FHE.pk}, \text{FHE.Enc}(\text{FHE.sk}, \text{PKE.sk}))$ is not linkable to any PKE.sk , so all these schemes are strongly-detection-key-unlinkable. $\square$

Remark 3.7. OMRt1 [58, Alg 5] does not satisfy our strong-detection-key-unlinkability (Definition 3.5) because $\text{sk}_{\text{clue}} = \text{sk}_{\text{det}} = \text{FHE.sk}$. However, this can be easily fixed essentially without any overhead by having $\text{sk}_{\text{clue}} = \text{FHE.sk}_1$ and $\text{sk}_{\text{det}} = \text{FHE.sk}_2$ by sampling two different FHE secret keys. Then, $\text{pk}_{\text{det}} = \text{FHE.Enc}(\text{sk}_2, \text{sk}_1)$ (encrypting sk_1 under sk_2). Then, replacing the $\text{FHE.Recrypt}(\text{pk}_{\text{det}}, c_{i,j})$ in [58, Alg 1] with $\text{FHE.Eval}(c_{i,j}, \text{Dec}_{\text{sk}_1})$ (i.e., homomorphically decrypting the clue ciphertexts under sk_i which is encrypted in pk_{det}).

Furthermore, as mentioned, this construction is only a feasibility result. It serves as a stepping stone to its main constructions, OMRp1, OMRp2, which both satisfy this property, as shown in the proof above.

4 Preliminary

4.1 Notation

Let $[n]$ denote the set $\{0, \dots, n-1\}$. For a vector $\vec{v}$, $\vec{v}[i]$ indicates the i -th element of this vector. Let $\mathcal{R} = \mathbb{Z}[X]/(X^D + 1)$ denote the $2D$ -th cyclotomic ring where D is a power-of-two (and here $[X]$ denotes “over variable X ”), and $\mathcal{R}_Q = \mathcal{R}/Q\mathcal{R}$ for some $Q \in \mathbb{Z}$ (i.e., for an element $x \in \mathcal{R}_Q$, x is a degree D polynomial with its coefficients in $\mathbb{Z}_Q$). A ring element $a \in \mathcal{R}_Q$ is represented as a polynomial of degree at most $D-1$ and each coefficient being $\in \mathbb{Z}_Q$: i.e., $a := a(X) = \sum_{i \in [D]} a_i X^i$ where $a_i \in \mathbb{Z}_Q$ are the coefficients for all $i \in [D]$. And let $a[i]$ denote the i -th coefficient of a . Additionally, let $\vec{a} \leftarrow \text{RingToVec}(a) \in \mathbb{Z}_Q^N$ denote the process of taking all the coefficients of $a \in \mathcal{R}_Q$ to form a vector, satisfying $\vec{a}[i] = a[i]$. Drawing x uniformly at random from a set S is denoted $x \leftarrow_{\$} S$. $T(\text{Procedure})$ means the runtime of a (potentially interactive) procedure **Procedure** and $|x|$ denotes the bit-size of an object x .

For a ring element $y = \sum_i y_i X^i \in \mathcal{R}$, let $y[i]$ denotes its i -th coefficient (i.e., y_i). Then, let $\vec{y}$ denotes a vector of all its coefficient, i.e., $\vec{y} := (y_0, \dots, y_{D-1}) \in \mathbb{Z}^{D \times 1}$. For a matrix A , $A[i][j]$ indicates the cell at the i -th row and j -th column. Furthermore, let $A[:\ell]$ denotes the first ℓ rows.

For a w -round protocol for $w \geq 1$, let procedure_i be the algorithm for the i -th round for $i \in [w]$; we use procedure to denote the collection of procedure_i in all rounds, and $T(\text{procedure}(\dots)) := \sum_{i \in [w]} T(\text{procedure}_i(\dots))$, where $T(\text{procedure}_i(\dots))$ is the runtime for procedure_i . Similarly, for item_i , the input or output of procedure_i , we use item to denote it in all w rounds. We use $|\text{item}| := \sum_{i \in [w]} |\text{item}_i|$ where $|\text{item}_i|$ is the size of item_i .³

4.2 Hardness Assumption

Definition 4.1 (Decisional ring learning with error assumption [65]). Let $D, Q, \mathcal{D}, \chi$ be parameters dependent on λ and D being a power of two. Let $\mathcal{R} = \mathbb{Z}[X]/(X^D + 1)$. The ring learning with error (RLWE) assumption $\text{RLWE}_{D,Q,\mathcal{D},\chi}$ is the following: for any PPT adversary, distinguishing $(a, a \cdot s + e)$ and (a, b) (with noticeable advantage) is hard, where $a \leftarrow_{\$} \mathcal{R}_Q, s \leftarrow \mathcal{D}, e \leftarrow \chi$ and $b \leftarrow_{\$} \mathcal{R}_Q$.

4.3 Private Information Retrieval

Private information retrieval (PIR) [26] is a primitive that, given a server holding a database $\text{DB} \in \mathbf{E}^N$ (where $\mathbf{E}$ is the space of a data entry, e.g., bit strings), a client can use PIR to retrieve $\text{DB}[i]$ without letting the server know any information about i . It is formally captured as follows.

Definition 4.2 (w -round Private information retrieval (PIR)). A Private Information Retrieval consists of three PPT procedures:

- $\text{pp} := (\text{pp}^S, \text{pp}^C) \leftarrow \text{GenParam}(1^\lambda, N)$: takes a security parameter λ , a database size N , and outputs a public parameter pp containing pp^S for the server and a public parameter pp^C for the client. Note that both pp^S, pp^C are available to the adversary for security. This distinction is only for efficiency reasons, as some public parameters may only be necessary to one of the two parties.

³The size is bit size for a general variable, and is the number of items if the variable denotes vector, such as the database BB .

PIR-SEC $_{\text{PIR},N}^{\mathcal{A}}(1^\lambda)$	
1 :	$\text{pp} = (\text{pp}^S, \text{pp}^C) \leftarrow \text{GenParams}(1^\lambda, N)$
2 :	$(\text{idx}_0 \in [N], \text{idx}_1 \in [N], \text{st}) \leftarrow \mathcal{A}(\text{pp}, N)$
3 :	$b \xleftarrow{\$} \{0, 1\}$
4 :	$\text{qry}_0, \text{st}_0 \leftarrow \text{Query}(\text{pp}^C, \text{idx}_b)$
5 :	for $i \in [w]$:
6 :	$\text{ans}_i, \text{st} \leftarrow \mathcal{A}(\text{pp}, \text{st}, \text{qry}_0)$
7 :	$\text{qry}_{i+1}, \text{st}_{i+1} \leftarrow \text{Decode}_i(\text{pp}, \text{ans}_i, \text{st}_i)$
8 :	$b' \leftarrow \mathcal{A}(\text{st})$
9 :	return $b = b'$

Figure 5: PIR-SEC: security game for PIR privacy.

- $(\text{qry}_0, \text{st}_0) \leftarrow \text{Query}(\text{pp}, \text{idx})$: takes a public parameter pp , an index $\text{idx} \in [N]$ and outputs a query qry_0 and a state st_0 for the first round below.
- Data retrieval is a w -round protocol between the server and the client for $w \geq 1$, and each round $i \in [w]$ includes the following two procedures:
 - $\text{ans}_i \leftarrow \text{Ans}_i(\text{pp}^S, \text{qry}_i, \text{DB})$: takes a public parameter (for server) pp^S , a query qry_i for round i , and a database $\text{DB} \in \mathcal{E}^N$ (for a data entry space $\mathcal{E}$, e.g., bit strings), and output an answer ans_i .
 - $\text{st}_{i+1}, \text{qry}_{i+1} \leftarrow \text{Decode}_i(\text{pp}^C, \text{ans}_i, \text{st}_i)$: takes a public parameter (for client) pp^C , an ans_i , and state st_i in round i and outputs a secret state st_{i+1} and query qry_{i+1} for the next round.

The scheme must satisfy the following properties:

- (Correctness) For any $\lambda > 0, N = \text{poly}(\lambda), \text{DB} \in \mathcal{E}^N$ where $\mathcal{E} := \{0,1\}^{\text{poly}(\lambda)}$ is the entry space. Let $\text{pp}^S, \text{pp}^C \leftarrow \text{GenParam}(1^\lambda, N)$. For any $\text{idx} \in [N]$, let $(\text{qry}_0, \text{st}_0) \leftarrow \text{Query}(\text{pp}^C, \text{idx})$, and for $i \in [w]$, $\text{ans}_i \leftarrow \text{Ans}_i(\text{pp}^S, \text{qry}_i, \text{DB})$, and $\text{st}_{i+1}, \text{qry}_{i+1} \leftarrow \text{Decode}_i(\text{pp}^C, \text{ans}_i, \text{st}_i)$, it holds that $\Pr[\text{DB}[\text{idx}] \in \text{st}_w] \geq 1 - \text{negl}(\lambda)$.
- (Privacy) For any $\lambda > 0, N = \text{poly}(\lambda)$, for any PPT adversary $\mathcal{A}$: It holds that

$$\left| \Pr[\text{PIR-SEC}_{\text{PIR},N}^{\mathcal{A}}(1^\lambda) = 1] \right| \leq 1/2 + \text{negl}(\lambda) .$$

where PIR-SEC is defined in Fig. 5.

Batch PIR. Batch PIR allows the client to query $\text{bN} \leq N$ indices $S := \{i_1, \dots, i_{\text{bN}}\} \subset [N]$, instead of a single index $i \in [N]$, and returns $\text{DB}[i_1], \dots, \text{DB}[i_{\text{bN}}]$. Formally, we capture it as follows.

Definition 4.3 (Batched private information retrieval (batchPIR)). A Batched Private Information Retrieval consists of three PPT procedures:

- $\text{pp} := (\text{pp}^S, \text{pp}^C) \leftarrow \text{GenParam}(1^\lambda, N)$: takes a security parameter λ , a database size N , and outputs a public parameter pp containing pp^S for the server and a public parameter pp^C for the client. Note that both pp^S, pp^C are available to the adversary for security. This distinction is only for efficiency reasons, as some public parameters may only be necessary to one of the two parties.
- $(\text{qry}, \text{st}) \leftarrow \text{Query}(\text{pp}^C, \text{id}_{x_1}, \dots, \text{id}_{x_{bN}})$: takes a public parameter (for client) pp^C , indices $\text{id}_{x_1}, \dots, \text{id}_{x_{bN}} \subset [N]$ and outputs a query qry and a state st .
- Data retrieval is a w -round protocol between the server and the client for $w \geq 1$, and each round $i \in [w]$ includes the following two procedures:
 - $\text{ans}_i \leftarrow \text{Ans}_i(\text{pp}^S, \text{qry}_i, \text{DB})$: takes a public parameter (for server) pp^S , a query qry_i for round i , and a database $\text{DB} \in \mathcal{E}^N$ (for a data entry space $\mathcal{E}$, e.g., bit strings), and output an answer ans . This procedure can be interactive between the server and the client.
 - $\text{st}_{i+1}, \text{qry}_{i+1} \leftarrow \text{Decode}_i(\text{pp}^C, \text{ans}_i, \text{st}_i)$: takes a public parameter (for client) pp^C , an ans_i , and state st_i in round i and outputs a secret state st_{i+1} and query qry_{i+1} for the next round.

The scheme must satisfy the following properties:

- (Correctness) For any $\lambda > 0$, $N = \text{poly}(\lambda)$, $\text{DB} \in \mathcal{E}^N$ where $\mathcal{E} := \{0,1\}^{\text{poly}(\lambda)}$ is an entry space. Let $\text{pp}^S, \text{pp}^C \leftarrow \text{GenParam}(1^\lambda, N)$. For any $S := (\text{id}_{x_1}, \dots, \text{id}_{x_{bN}}) \subset [N]$, let $(\text{qry}_0, \text{st}_0) \leftarrow \text{Query}(\text{pp}^C, \text{id}_{x_1}, \dots, \text{id}_{x_{bN}})$, $\text{ans}_i \leftarrow \text{Ans}_i(\text{pp}^S, \text{qry}_i, \text{DB})$, and $\text{st}_{i+1}, \text{qry}_{i+1} \leftarrow \text{Decode}_i(\text{pp}^C, \text{ans}_i, \text{st}_i)$, it holds that $\Pr[(\forall i \in [w], \text{DB}[i] \in \text{st}_w)] \geq 1 - \text{negl}(\lambda)$.
- (Privacy)

$$\left| \Pr[\text{BATCHPIR-SEC}_{\text{PIR}, N}^A(1^\lambda) = 1] \right| \leq 1/2 + \text{negl}(\lambda) .$$

where BATCHPIR-SEC is defined in Fig. 6.

It is well known that PIR implies batchPIR [4, 3, 22], with constant overhead, captured formally in the following lemma, and see [4] for the closed-box compiler:

Lemma 4.4. *Let Π be a w -round PIR scheme (Definition 4.2), there exists π' that is a w -round batchPIR scheme (Definition 4.3) satisfying $T(\pi'.\text{Query}(\cdot, S)) = \tilde{O}(T(\pi'.\text{Query}(\cdot, \cdot)))$ for any $S \subseteq [N]$, and $|\pi'.\text{qry}| = \tilde{O}(|\pi.\text{qry}|)$, $|\pi'.\text{ans}| = \tilde{O}(|\pi.\text{ans}|)$.*

4.4 Public Key Encryption

Definition 4.5 (Public Key Encryption (PKE)). A PKE scheme consists of four PPT algorithms (GenParam , KeyGen , Enc , Dec) defined as follows:

- $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$: takes a security parameter λ ; outputs a public parameter pp .
- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$: takes the public parameter pp ; outputs a key pair (pk, sk) .

BATCHPIR-SEC _{PIR,N} ^A (1 ^λ)
1 : $\text{pp} = (\text{pp}^S, \text{pp}^C) \leftarrow \text{GenParams}(1^\lambda, N)$
2 : $(bN \in [N], S_0 \in [N]^{bN}, S_1 \in [N]^{bN}, \text{st}) \leftarrow \mathcal{A}(\text{pp}, N)$
3 : $b \xleftarrow{\$} \{0, 1\}$
4 : $\text{qry}_0, \text{st}_0 \leftarrow \text{Query}(\text{pp}^C, S_b)$
5 : for $i \in [w]$:
6 : $\text{ans}_i, \text{st} \leftarrow \mathcal{A}(\text{pp}, \text{st}, \text{qry}_0)$
7 : $\text{qry}_{i+1}, \text{st}_{i+1} \leftarrow \text{Decode}_i(\text{pp}, \text{ans}_i, \text{st}_i)$
8 : $b' \leftarrow \mathcal{A}(\text{st})$
9 : return $b = b'$

Figure 6: BATCHPIR-SEC: security game for batch PIR privacy.

- $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{pk}, m)$: takes the public parameter pp and the public key pk , generates the ciphertext c for the message m ; outputs ct .
- $m \leftarrow \text{Dec}(\text{pp}, \text{sk}, \text{ct})$: takes the public parameter pp , decrypts the ciphertext ct into a plaintext message m based on the secret key sk ; outputs the message m .

The scheme must satisfy the following properties:

- (Correctness) For any $\lambda > 0$, let $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$, let $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$ and for any message m , let $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{pk}, m)$ and $m' \leftarrow \text{Dec}(\text{pp}, \text{sk}, \text{ct})$. It holds that if $\text{pp} \neq \perp$, $\Pr[m = m'] \geq 1 - \text{negl}(\lambda)$.
- (IND-CPA Security) For any PPT algorithm $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$: let $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$, $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$; $\mathcal{A}_1$ chooses two messages and remembers state: $(m_0, m_1, \text{st}) \leftarrow \mathcal{A}_1(\text{pp}, \text{pk})$; sample $b \xleftarrow{\$} \{0, 1\}$ and $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{pk}, m_b)$; let $b' \leftarrow \mathcal{A}_2(\text{st}, \text{ct})$; it holds that: $\Pr[b = b'] \leq 1/2 + \text{negl}(\lambda)$.

Then, we define two additional optional properties:

- (Key privacy [11]) A PKE scheme is key-private or has key-privacy property if: For any PPT algorithm $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$: let $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$, and $(\text{pk}_0, \text{sk}_0) \leftarrow \text{KeyGen}(\text{pp})$, $(\text{pk}_1, \text{sk}_1) \leftarrow \text{KeyGen}(\text{pp})$; $\mathcal{A}_1$ chooses a message and remembers its state: $(m, \text{st}) \leftarrow \mathcal{A}_1(\text{pp}, \text{pk}_0, \text{pk}_1)$; then sample $b \xleftarrow{\$} \{0, 1\}$ and let $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{pk}_b, m)$; let $b' \leftarrow \mathcal{A}_2(\text{st}, \text{ct})$; it holds that: $\Pr[b = b'] \leq 1/2 + \text{negl}(\lambda)$.
- (Wrong-key Decryption [58]) A PKE scheme has the wrong-key-decryption property if: For a PKE scheme with plaintext space $\mathcal{P}$, let $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$, $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$, $(\text{pk}', \text{sk}') \leftarrow \text{KeyGen}(\text{pp})$, for any $m \in \mathcal{P}$, let $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{pk}, m)$, $m' \leftarrow \text{Dec}(\text{pp}, \text{sk}', m)$, it holds that $\Pr[m' = m] \leq 1/|\mathcal{P}| + \text{negl}(\lambda)$.

The RLWE PKE scheme. Now, we recall the RLWE PKE scheme introduced in [65] (with the modifications used in [60]).

Definition 4.6 (RLWE PKE). RLWEenc has the following algorithms:

- $\text{pp} = (n, \ell, q, \sigma, r, \mathcal{D}) \leftarrow \text{RLWEenc.GenParam}(1^\lambda, \ell, q, \sigma, \epsilon_n)$: Choose a secret key dimension n (ring dimension, with ring $\mathbb{Z}[X]/(X^n + 1)$) and a distribution $\mathcal{D}$ where the distribution is sampling a random vector of form $\{-1, 0, 1\}^n$ with a fixed Hamming weight h , such that $\text{RLWE}_{n,q,\mathcal{D},\sigma}$ holds. Set ciphertext modulus q , number of bits in plaintext $\ell \leq n$, and standard deviation σ for Gaussian distribution for ciphertext noise generation. Additionally, set a minimum integer r such that $\text{erf}(\frac{r}{\sqrt{2} \cdot \sqrt{2h+1} \cdot \sigma}) \leq \epsilon_n/\ell$.
- $(\text{sk}, \text{pk}) \leftarrow \text{RLWEenc.KeyGen}(\text{pp})$: Draw a secret key $s \leftarrow \mathcal{D}$. Sample $\alpha \leftarrow_{\$} \mathcal{R}_q$ and noise $x \leftarrow \chi_\sigma \in \mathcal{R}$, and compute $\text{pk} = (\alpha, \alpha s + x) \in \mathcal{R}_q \times \mathcal{R}_q$, $\text{sk} \leftarrow s$.
- $\text{ct} = (a, \vec{b}) \leftarrow \text{RLWEenc.Enc}(\text{pp}, \text{pk}, \vec{m})$: To encrypt a vector $\vec{m} \in \mathbb{Z}_2^\ell$, define the ring element $t \leftarrow \sum_{i \in [\ell]} \frac{q}{2} \cdot \vec{m}[i] X^i \in \mathcal{R}_q$. Draw a ring element $y \leftarrow \mathcal{D} \in \mathcal{R}_q$ and noises $x', x'' \leftarrow \chi_\sigma$. Let $b \leftarrow \text{pk}[1] \cdot y + t + x'' = \sum_{i \in [n]} b_i X^i$. The ciphertext is the pair $(a, \vec{b}) = (\alpha \cdot y + x', (b_i)_{i \in [\ell]}) \in \mathcal{R}_q \times \mathbb{Z}_q^\ell$.
- $\vec{m} \leftarrow \text{RLWEenc.Dec}(\text{pp}, \text{sk}, \text{ct} = (a, \vec{b}))$: Let $a' \leftarrow a \cdot \text{sk} := \sum_{i \in [n]} a'_i X^{i-1}$, $\vec{d} = \vec{b} - (a'_i)_{i \in [\ell]}$, $\vec{m} = (\vec{d} > r) \in \mathbb{Z}_2^\ell$.

RLWEenc satisfies correctness, IND-CPA Security, key-privacy, and wrong-key-decryption (Section 4.4), guaranteed by the RLWE assumption (Section 4.2) [65].

4.5 Additively Homomorphic Encryption

Additively homomorphic encryption (AHE) further allows the ciphertexts to perform linear operations. In this work, we use Brakerski/Fan-Vercauteren (BFV) homomorphic encryption scheme [21, 33] as our AHE, which is technically fully homomorphic encryption (allowing arbitrary computation over ciphertexts [81, 37]), but we only use its additive homomorphism.

BFV scheme consists of the following PPT algorithms: $\text{GenParam}(1^\lambda), \text{KeyGen}(\text{pp}_{\text{BFV}}), \text{Enc}(\text{pp}_{\text{BFV}}, \text{pk}, m), \text{Dec}(\text{pp}_{\text{BFV}}, \text{sk}, c)$ as normal PKE schemes. BFV is unconditionally correct. Under the *RLWE* hardness assumption (Section 4.2), it also fulfills the standard definitions of semantic security (IND-CPA) for FHE schemes.

Given a polynomial from the cyclotomic ring $\mathcal{R}_t = \mathbb{Z}_t[X]/(X^D + 1)$ (where D is a power-of-two, $t \equiv 1 \pmod{2D}$), the BFV scheme encrypts it into a ciphertext consisting of two polynomials, each of which in a larger cyclotomic ring $\mathcal{R}_Q = \mathbb{Z}_Q[X]/(X^D + 1)$ for some $Q > t$. Here, t , Q , and D are called the plaintext modulus, the ciphertext modulus, and the ring dimension, respectively.

Alternatively, one can also encode a plaintext in $\mathbb{Z}_t^D$ into a polynomial in $\mathcal{R}_t$ and then encrypt (see Appendix B for details). In this case, each $\mathbb{Z}_t$ element is encoded into a “slot” of the ring element, which contains D slots in total. This ability to evaluate D of $\mathbb{Z}_t$ elements simultaneously is the so-called Single-Instruction-Multiple-Data (SIMD) property of BFV.

Linear operations. While BFV is indeed an FHE scheme, as mentioned, we only use the linear operations. Specifically, we use the following operations:

- (Addition) For any two BFV ciphertexts ct_1, ct_2 , let $\text{ct} \leftarrow \text{ct}_1 + \text{ct}_2$, it holds that $\text{BFV.Dec}(\text{ct}) = \text{BFV.Dec}(\text{ct}_1) + \text{BFV.Dec}(\text{ct}_2)$.

- (Plaintext Multiplication) For any BFV ciphertext ct and plaintext $m \in \mathcal{R}_t$, let $\text{ct}' \leftarrow m \times \text{ct}$, it holds that $\text{BFV.Dec}(\text{ct}') = m \times \text{BFV.Dec}(\text{ct})$.
- (Matrix Multiplication) For any BFV ciphertext ct and matrix $M \in \mathbb{Z}_t^{M \times D}$ for any $M \leq D$, let $\text{ct}' \leftarrow M \times \text{ct}$, it holds that $\text{BFV.Dec}(\text{ct}') = M \times \text{BFV.Dec}(\text{ct})$, where the multiplication is standard matrix-vector multiplication over $\mathbb{Z}_t$ for BFV.Dec decrypting into $\mathbb{Z}_t^D$ instead of $\mathcal{R}_t$.

5 The Relationship Between OMR and PIR

As discussed in Section 2.1, both OMR and PIR enable a recipient to retrieve database records without revealing the access pattern. However, the key distinction is that OMR achieves this using encrypted, random-looking clues, whereas PIR typically relies on plaintext indices or keywords. This similarity and difference hint that OMR provides a stronger functionality than PIR. This section formalizes this intuition.

5.1 OMR Implies PIR

This section presents a closed-box reduction from PIR to any OMR construction with strong detection-key-unlinkability (Definition 3.5). Furthermore, this reduction has almost no efficiency overhead: The online runtime of the PIR server matches that of the OMR detector when the payload is the PIR database. Moreover, the PIR online communication is the detection key size plus the digest size, which is also the normal online communication of the OMR construction.⁴

Theorem 5.1. *Let OMR be a w -round OMR scheme (Definition 3.1), which is also strong detection-key-unlinkable (Definition 3.5). Then, there exists a w -round PIR scheme PIR (Definition 4.2) with the following efficiency guarantee (see Section 4.1 for notations):*

- $T(\text{PIR.Ans}(\text{pp}_{\text{PIR}}^S, \text{qry}, \text{DB})) = T(\text{OMR.Retrieve}(\text{pp}_{\text{OMR}}, \text{BB}, \text{qry}, 1, \text{aux}))$, where $\text{pp}_{\text{PIR}}^S, \text{qry}, \text{BB}, \text{pp}_{\text{OMR}}$ are generated as in the proof below, and $\text{DB} \in \mathcal{E}^N$ is the PIR database, satisfying $|\text{BB}.x| \leq 2|\text{DB}|$ for $\log(|\mathcal{E}|) \geq \log(N)$.
- $T(\text{PIR.Decode}(\text{pp}_{\text{PIR}}^C, \text{ans}, \text{st})) = T(\text{OMR.Decode}(\text{pp}_{\text{OMR}}, \text{ans}, \text{sk}, \text{stR}))$, where $\text{pp}_{\text{PIR}}^C, \text{ans}, \text{st}, \text{pp}_{\text{OMR}}, \text{sk}$ are generated as in the proof below.
- $|\text{PIR.qry}| = |\text{OMR.pk}_{\text{det}}| + |\text{OMR.aux}|$.
- $|\text{PIR.ans}| = |\text{OMR.dig}|$.

Proof. Given OMR, we construct the PIR scheme PIR as follows, where $i \in [w]$ is the algorithm in round i :

- $\text{PIR.GenParam}(1^\lambda, N)$:
 - $\text{pp}_{\text{OMR}} \leftarrow \text{OMR.GenParam}(1^\lambda, \text{negl}(\lambda), \text{negl}(\lambda))$;
 - Sample a random seed s ;

⁴Technically, OMR recipients could submit a detection key offline, and this key can be reused for different bulletin boards.

- For $j \in [N]$, $(\text{sk}_{\text{clue}j}, \text{pk}_{\text{clue}j}) \leftarrow \text{OMR.GenClueKey}(\text{pp}; G(s, j))$, $c_j \leftarrow \text{OMR.GenClue}(\text{pp}_{\text{OMR}}, \text{pk}_{\text{clue}j})$, where G is a PRG which takes a seed s and an index j and outputs a randomness deterministically;
- Return $\text{pp}_{\text{PIR}} = (\text{pp}_{\text{PIR}}^S := (\text{pp}_{\text{OMR}}, s, \{c_j\}_{j \in [N]}), \text{pp}_{\text{PIR}}^C := (\text{pp}_{\text{OMR}}, s))$.
- $\text{PIR.Query}(\text{pp}_{\text{PIR}}^C, \text{idx})$:
 - $\text{sk}_{\text{clueidx}}, \text{pk}_{\text{clueidx}} \leftarrow \text{OMR.GenClueKey}(\text{pp}_{\text{OMR}}; G(s, \text{idx}))$;
 - $\text{sk}_{\text{det}}, \text{pk}_{\text{det}} \leftarrow \text{OMR.GenDetKey}(\text{pp}_{\text{OMR}}, \text{sk}_{\text{clueidx}})$ (generated with a fresh randomness);
 - Return $\text{qry}_0 := \text{pk}_{\text{det}}, \text{st}_0 := \text{sk} := (\text{sk}_{\text{clue}}, \text{sk}_{\text{det}})$.
- $\text{PIR.Ans}_i(\text{pp}_{\text{PIR}}^S, \text{qry}_i, \text{DB})$:
 - Let $\text{aux}_{i-1} \leftarrow \perp$ if $i = 0$ and $\text{aux}_{i-1} \leftarrow \text{qry}_i$ otherwise;
 - Let $\text{BB} := \{c_j, \text{DB}[j] || j\}_{j \in [N]}$;
 - $\text{stR}_i, \text{aux}_i \leftarrow \text{OMR.Retrieve}(\text{pp}_{\text{OMR}}, \text{BB}, \text{qry}_0, \bar{k} = 1, \text{aux}_{i-1})$;
 - Return $\text{PIR.st}_{i+1} = \text{stR}_i, \text{qry}_{i+1} = \text{aux}_i$.
- $\text{PIR.Decode}_i(\text{pp}_{\text{PIR}}^C, \text{ans}, \text{st}_i)$:
 - $\text{stR}_{i+1}, \text{aux}_{i+1} \leftarrow \text{OMR.Decode}_i(\text{pp}_{\text{OMR}}, \text{ans}, \text{sk} := (\text{sk}_{\text{clue}j}, \text{sk}_{\text{det}}))$;
 - Return $\text{st}_{i+1} := \text{stR}_{i+1}, \text{qry}_{i+1} = \text{aux}_{i+1}$.

Now, we prove the properties of PIR.

Correctness: By the OMR completeness, since there is exactly one message generated using $\text{pk}_{\text{clue}i}$ and all the messages are unique (since each database entry is appended by the unique index i), it holds that $\text{DB}[\text{idx}] || \text{idx} \in \text{PL}$ (except for negligible probability). Therefore, $\text{DB}[\text{idx}] \in \text{st}_w$ as required by PIR (except for negligible probability).

Privacy: We prove privacy via a series of hybrids, corresponding to the protocols integrated in the security game:

- hyb_0 : our PIR above.
- hyb_1 : instead of storing the seed s in pp_{PIR} , stores all the $G(s, \text{idx})$ for $\text{idx} \in [N]$.
- hyb_2 : $\text{sk}_{\text{clue}i}, \text{pk}_{\text{clue}i}$'s are generated via true randomness instead of $G(s, i)$.

It is easy to see that privacy-wise, hyb_0 and hyb_1 are trivially identical. Then, hyb_1 and hyb_2 have the same privacy guarantee by the security of the PRG G and seed s being correctly sampled. Thus, it remains to show that hyb_2 is indeed a secure PIR construction.

We prove the privacy guarantee of hyb_2 via a reduction. Suppose there exists an adversary $\mathcal{A}$ that breaks PIR for some $N \geq 2$, we can build $\mathcal{A}'$ to break the strong-detection-key-unlinkability as follows.

Denote the input of $\mathcal{A}'$ as $(\text{pp}_{\text{OMR}}, r_0, r_1, \text{pk}_{\text{det}b}, \bar{k} = 1)$, where $b \in \{0, 1\}$ is hidden from $\mathcal{A}'$. Generate pp_{PIR} honestly, where each clue key pair is generated using randomness r'_i for $i \in [N]$. Randomly sample two indices $\text{idx}'_0, \text{idx}'_1 \in [N]$ and replace $r'_{\text{idx}'_j}$ with r_j for $j \in \{0, 1\}$. Obtain $\text{idx}_0, \text{idx}_1$ from $\mathcal{A}$. Return a random bit b' directly if $(\text{idx}_0, \text{idx}_1)$ do not match $(\text{idx}'_0, \text{idx}'_1)$. Otherwise,

answer $\mathcal{A}$ by using $\text{pk}_{\text{det}b}$ by playing the OMR-SDKU game, and return whatever $\mathcal{A}$ returns. This is equivalent to playing the PIR-SEC game with $\mathcal{A}$ using $\text{id}x_b$, so the advantage of winning the OMR-SDKU game is at least $1/N^2$ times the advantage of $\mathcal{A}$ in winning the PIR-SEC game.

Efficiency: Lastly, we discuss the efficiency, which is straightforward:

- $T(\text{PIR}.\text{Ans}(\text{pp}_{\text{PIR}}^S, \text{qry}, \text{DB})) = T(\text{OMR}.\text{Retrieve}(\text{pp}_{\text{OMR}}, \text{BB}, \text{qry}, 1, \text{aux}))$ (plus the runtime to pad DB entries with their indices, but, for simplicity, we view that as 0 since it can be done when passing the padded DB to $\text{OMR}.\text{Retrieve}$).
- $T(\text{PIR}.\text{Decode}(\text{pp}_{\text{PIR}}^C, \text{ans}, \text{st})) = T(\text{OMR}.\text{Decode}(\text{pp}_{\text{OMR}}, \text{ans}, \text{sk}, \text{st}))$ since $\text{PIR}.\text{Decode}$ only calls $\text{OMR}.\text{Decode}$ without doing anything else.
- $|\text{PIR}.\text{qry}| = |\text{OMR}.\text{pk}_{\text{det}}|$ by our construction above.
- $|\text{PIR}.\text{ans}| = |\text{OMR}.\text{dig}|$ by our construction above.

□

An alternative reduction. There is an alternative approach to perform the reduction: instead of generating the clues c_i 's during pp_{PIR} generation, the server generates them during the $\text{PIR}.\text{Ans}$ procedure using the seed s . The rest of the reduction remains exactly the same. The change of efficiency is that now $T(\text{PIR}.\text{Ans}) = T(\text{OMR}.\text{Retrieve}) + N \cdot (T(\text{OMR}.\text{KeyGen}) + T(\text{OMR}.\text{GenClue}))$, while $|\text{pp}_{\text{PIR}}^S|$ is reduced from $O(N)$ to $O(1)$. We believe that our reduction in the current proof is more efficient since there is essentially *no* online runtime loss, which is one of the two main efficiency metrics PIR literature try to improve upon (with the other one being online communication) [4, 3, 68]. On the other hand, since pp_{PIR}^S is only held by the server, who holds a database of size N already, storing pp_{PIR}^S does not seem to be a large efficiency loss.

BatchPIR. While our proof above does not directly show that OMR implies batchPIR, by Lemma 4.4, OMR can be used to build batchPIR directly as well, with constant overhead. However, how to show OMR implies batchPIR (with less overhead) in a more direct way is an interesting open problem.

Larger ϵ_n . While an OMR construction should support any $\epsilon_n, \epsilon_p \in (0, 1)$, one could weaken the OMR definition to only allow certain range of ϵ_p, ϵ_n (e.g., only non-negligible in λ). PIR can still be constructed (with slightly larger overhead). In more detail, the server can simply call Retrieve for $w = \log_{\epsilon_n}(\text{negl}(\lambda))$ times using independently generated clues. In this case, each call has a failure probability of ϵ_n and all of them fail with probability $\epsilon_n^w = \text{negl}(\lambda)$ times. This achieves PIR with runtime efficiency $T(\text{PIR}.\text{Ans}) = w \cdot T(\text{OMR}.\text{Retrieve})$, $T(\text{PIR}.\text{Decode}) = w \cdot T(\text{OMR}.\text{Decode})$, and $|\text{PIR}.\text{ans}| = w \cdot |\text{OMR}.\text{dig}|$, while $|\text{PIR}.\text{Query}|$ remains unchanged. Asymptotically, this adds little overhead (at most λ). Concretely, for OMR, a larger ϵ_n makes the primitive moot and therefore even if ϵ_n is not negligible, it should be small enough and thereby w being a very small constant.

Note that all the OMR constructions indeed support $\text{negl}(\lambda)$ of ϵ_n, ϵ_p as required by the definition itself. We only include this discussion for completeness.

[P3] The $1/N^2$ loss. As one may keenly observe, the reduction above incurs a security loss of $1/N^2$. Under the current security definition, it appears difficult to eliminate this loss. Indeed, one could weaken the security notion by, for example, requiring the adversary to commit to its choice of indices before seeing the public parameters, which removes the $1/N^2$ loss. However, this modification fails to capture real-world scenarios in which public parameters are shared in advance.

This limitation also explains why we are unable to directly reduce OMR to batch PIR: For k indices, the natural extension of the above reduction incurs a loss of $\binom{N}{k}$, which can be super-polynomial in the security parameter. Removing such a loss to achieve a tighter reduction is an interesting open question.

5.2 OMR Implies Key-private PKE

This section then shows that OMR implies key-private PKE (with wrong-key-decryption). As mentioned, while this directly shows OMR is a strictly harder primitive than PIR, this result means that if PIR implies OMR (since single-server PIR implies PKE [31, 38]), additively homomorphic symmetric key encryption (AHSKE) implies key-private PKE (see Remark C.3 for a more detailed discussion).⁵

In fact, this section takes a step further and shows that (non-compact) OMR (Definition 3.1) is *equivalent* to PKE with key-privacy and wrong-key-decryption property (Definition 4.5).

Theorem 5.2. *There exists an OMR construction (Definition 3.1), if and only if there exists a PKE construction that has key-privacy and wrong-key-decryption property (Definition 4.5) with message space $\{0,1\}$.*

Proof. We first show that given OMR, we can construct a PKE scheme PKE as follows (for simplicity, we assume OMR is one-round and drop the subscript, but the construction below trivially extends to multiple rounds, since all the rounds can be simulated by the recipient holding the secret key):

- $\text{PKE.GenParam}(1^\lambda)$:
 - $\text{pp}_{\text{OMR}} \leftarrow \text{OMR.GenParam}(1^\lambda, \text{negl}(\lambda), \text{negl}(\lambda))$
 - Return $\text{pp}_{\text{PKE}} := \text{pp}_{\text{OMR}}$
- $\text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$:
 - For $i \in \{0,1\}$: $\text{sk}_{\text{clue}_i}, \text{pk}_{\text{clue}_i} \leftarrow \text{GenClueKey}(\text{pp}_{\text{OMR}})$
 - For $i \in \{0,1\}$: $\text{sk}_{\text{det}_i}, \text{pk}_{\text{det}_i} \leftarrow \text{GenDetKey}(\text{pp}_{\text{OMR}}, \text{sk}_{\text{clue}_i})$
 - Return $\text{sk} := (\text{sk}_{\text{clue}_0}, \text{sk}_{\text{det}_0}, \text{pk}_{\text{det}_0}, \text{sk}_{\text{clue}_1}, \text{sk}_{\text{det}_1}, \text{pk}_{\text{det}_1})$, $\text{pk} := (\text{pk}_{\text{clue}_0}, \text{pk}_{\text{clue}_1})$
- $\text{PKE.Enc}(\text{pp}_{\text{PKE}}, \text{pk}, m \in \{0,1\})$:
 - $r \leftarrow_{\$} \{0,1\}^\lambda$
 - $c \leftarrow \text{GenClue}(\text{pp}_{\text{OMR}}, \text{pk}_{\text{clue}_b})$
 - Return $\text{ct} := (c, r)$
- $\text{PKE.Dec}(\text{pp}_{\text{PKE}}, \text{sk}, \text{ct})$:
 - $\text{dig} \leftarrow \text{OMR.Retrieve}(\text{pp}_{\text{OMR}}, \text{pk}_{\text{det}_0}, \{\text{ct}\})$
 - $\text{PL}_0 \leftarrow \text{OMR.Decode}(\text{pp}_{\text{OMR}}, \text{ans}, (\text{sk}_{\text{clue}_0}, \text{sk}_{\text{det}_0}))$
 - $\text{dig} \leftarrow \text{OMR.Retrieve}(\text{pp}_{\text{OMR}}, \text{pk}_{\text{det}_1}, \{\text{ct}\})$

⁵It is also not known whether AHSKE implies PKE with wrong-key-decryption, but this property is less standard and quite new.

- $\text{PL}_1 \leftarrow \text{OMR.Decode}(\text{pp}_{\text{OMR}}, \text{ans}, (\text{sk}_{\text{clue1}}, \text{sk}_{\text{det1}}))$
- If $r \in \text{PL}_0$, return 0. If $r \in \text{PL}_1$, return 1. Otherwise, return a random bit.

Now, we prove the properties of PKE.

- Correctness: By the completeness of OMR, if $m = 0$, $r \in \text{PL}_0$ with probability $\geq 1 - \text{negl}(\lambda)$. By the soundness of OMR, if $m = 1$, $r \notin \text{PL}_0$ with probability $\geq 1 - \text{negl}(\lambda)$. However, again by completeness, $r \in \text{PL}_1$ with probability $\geq 1 - \text{negl}(\lambda)$. Thus, the correctness of PKE holds.
- CPA-security: If there exists an adversary $\mathcal{A}$ that breaks the CPA security of PKE, it trivially distinguishes the clue generated using pk_{clue0} and the clue generated using pk_{clue1} , as that is the only difference for ct. Thus, PKE is CPA-secure.
- Key-privacy: Let $\text{pk} = (\text{pk}_{\text{clue0}}, \text{pk}_{\text{clue1}})$ and $\text{pk}' = (\text{pk}'_{\text{clue0}}, \text{pk}'_{\text{clue1}})$ be two honestly generated PKE public keys. If there exists an adversary $\mathcal{A}$ that breaks the key privacy of PKE, it means that this $\mathcal{A}$ distinguishes the clue generated using $\text{pk}_{\text{clue}b}$ and the clue generated using $\text{pk}'_{\text{clue}b}$ for some $b \in \{0,1\}$, which the privacy guarantee of OMR. Thus, PKE is key-private.
- Wrong-key-decryption: By the soundness of OMR, since the clue is generated via another (honestly generated) public key, both Retrieve using $\text{pk}_{\text{det0}}, \text{pk}_{\text{det1}}$ would not be able to successfully include r in PL_i for $i \in [2]$ (except for negligible probability). Thus, the decryption procedure returns a random value. This satisfies the definition of wrong-key-decryption.

Then, we show the other direction: Given a PKE scheme PKE with key-privacy and wrong-key-decryption property, we can construct a (non-compact) OMR scheme OMR as follows:

- $\text{OMR.GenParam}(1^\lambda, \epsilon_n, \epsilon_p)$:
 - $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.GenParam}(1^\lambda)$
 - $\ell \leftarrow \lceil -\log(\epsilon_p) \rceil$
 - Return $\text{pp}_{\text{OMR}} := (\text{pp}_{\text{PKE}}, \ell)$
- $\text{OMR.GenClueKey}(\text{pp}_{\text{OMR}})$:
 - $(\text{pk}, \text{sk}) \leftarrow \text{PKE.KeyGen}(\text{pp.PKE})$
 - Return $\text{sk}_{\text{clue}} = \text{sk}, \text{pk}_{\text{clue}} = \text{pk}$
- $\text{OMR.GenDetKey}(\text{pp}_{\text{OMR}}, \text{sk}_{\text{clue}})$:
 - Return $\text{sk}_{\text{det}} = \text{sk}_{\text{clue}}, \text{pk}_{\text{det}} = \perp$
- $\text{OMR.GenClue}(\text{pp}_{\text{PKE}}, \text{pk}_{\text{clue}})$:
 - $\text{ct}_i \leftarrow \text{PKE.Enc}(\text{pp}_{\text{PKE}}, 0)$ for $i \in [\ell]$
 - Return $c = \{\text{ct}_i\}_{i \in [\ell]}$
- $\text{OMR.Retrieve}(\text{pp}_{\text{OMR}}, \text{pk}_{\text{det}}, \text{BB}, \bar{k})$:
 - Return $\text{dig} = \text{BB}$

- $\text{OMR.Decode}(\text{pp}_{\text{OMR}}, \text{sk}, \text{dig})$:
 - For all $i \in [N]$
 - If $\text{PKE.Dec}(\text{pp}_P KE, \text{sk}, \text{BB}[i].c[j]) = 0$ for all $j \in [\ell]$, include $\text{BB}[i].x \in \text{PL}$
 - Return PL

Now, we prove the properties of OMR.

- Completeness: By correctness of PKE, if the clue is encrypted under pk_{clue} , the corresponding sk_{clue} decrypts it to 0^ℓ . Thus, the corresponding payload is included in PL.
- Soundness: By wrong-key-decryption of PKE, if the clue is encrypted under a different honestly generated clue key pk'_{clue} , decryption under sk_{clue} (corresponding to pk_{clue}) result in 0^ℓ with $1/2^\ell \leq \epsilon_p$ probability.
- Privacy: Privacy is guaranteed by the key-privacy of PKE.

As a side note, this OMR is in fact trivially strongly-detection-key-unlinkable, simply because $\text{pk}_{\text{det}} = \perp$. $\square$

Larger ϵ_n, ϵ_p . Similar to the previous reduction from PIR to OMR, the reduction does not directly work if ϵ_n or ϵ_p cannot be $\text{negl}(\lambda)$ (which again is only a weaker version of OMR that could potentially be defined by future works and we discuss this for completeness only). Now we have $\epsilon_n = p$ and $\epsilon_p = p'$ such that $|(1-p) - p'| = \text{nonnegl}(\lambda)$ (note that if $1-p = p'$ means that pertinent messages and impertinent messages are indistinguishable, making the OMR construction moot). WLOG, assume $p < 1 - p'$. Then, we can simply repeat the encryption process above w times. Then, it means that if the encryption of 0, Retrieve using $\text{pk}_{\text{det}0}$ contains the dummy string r for $w \cdot (1-p)$ trials (in expectation). On the other hand, if the encryption is 1, Retrieve using $\text{pk}_{\text{det}1}$ contains the dummy string r for $w \cdot p'$ trials (in expectation). There is a non-negligible gap between $(1-p)$ and p' , which means that as long as $w = \text{poly}(\lambda)$ is large enough, we are able to tell whether the encryption is 0 or 1. However, again, this is only to show theoretical feasibility.

Compactness. Note that this proof shows the equivalence of OMR and key-private PKE with wrong-key-decryptin property, but only for non-compact OMR (i.e., $|\text{dig}| = \Omega(N)$, and see Appendix A for formal compactness definition). For a compact OMR, it is not directly implied by such a PKE scheme anymore.

6 The Main OMR Construction

This section introduces our main construction of oblivious message retrieval: UnifOMR. As mentioned in Section 2.2, all existing works on single-server OMR rely on (leveled) fully homomorphic encryption to achieve asymptotic compactness. However, this largely limits the efficiency of the resulting construction: It takes > 20 minutes to process 2^{19} messages. If, instead, we focus only on “concrete compactness” (i.e., digest size being concretely small but not asymptotically sublinear in database size), the detector runtime can be greatly improved by avoiding the need for FHE.

6.1 Overview of Retrieval based on Additively Homomorphic Encryption

This section starts with the basic idea of UnifOMR. In the following sections, we will discuss how to further optimize the scheme and present the full construction.

Clue key and clue generation. At the beginning, the recipients need to generate the clue keys. Similar to prior single-server OMR works [60, 51, 57], UnifOMR uses the PKE scheme RLWEenc (Definition 4.6) for clue generation.

Each recipient generates an RLWEenc key pair and uses them as the clue key pair ($\text{sk}_{\text{clue}} := \text{RLWEenc.sk}, \text{pk}_{\text{clue}} := \text{RLWEenc.pk}$). Then, to send a message to the recipient, the sender first fetches the recipient's public key pk_{clue} , and generates an RLWEenc ciphertext $c := (a, \vec{b}) \leftarrow \text{RLWEenc.Enc}(\text{pp}_{\text{RLWEenc}}, \text{pk}_{\text{clue}}, 0^\ell) \in \mathcal{R}_t \times \mathbb{Z}_t^\ell$ as the clue, which encrypts a vector of ℓ zeros. ℓ is some parameter adjusted according to ϵ_p to-be-discussed later, and t is the plaintext modulus for the BFV scheme.⁶

Pertinent indices detection via partial decryption. To help the recipient identify which messages are pertinent, the detector could perform a full homomorphic decryption of the clue ciphertext c , which is the method taken by all prior single-server OMR works. However, this is very inefficient since the decryption requires the homomorphic evaluation of the entire decryption circuit, which is deep and expensive.

Recall that for RLWEenc ciphertexts, the first step of decryption is a simple linear operation: $\vec{b} - \text{RingToVec}(\text{ask})$. To evaluate $\text{RingToVec}(\text{ask})$, a natural way is to view sk as a vector in $\mathbb{Z}_t^N$, i.e., $\vec{s} := \text{RingToVec}(\text{sk}) \in \mathbb{Z}_t^n$ (see Section 4.1 for notations), and then view $a = \sum_{i \in [n]} a_i X^i$ as a

$$\text{circulant matrix: } A := \begin{pmatrix} a_0 & -a_{n-1} & -a_{n-2} & \dots & -a_1 \\ a_1 & a_0 & -a_{n-1} & \dots & -a_2 \\ a_2 & a_1 & -a_0 & \dots & -a_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & a_{n-3} & \dots & a_0 \end{pmatrix} \in \mathbb{Z}_t^{n \times n}. \text{ Thus, } \text{RingToVec}(\text{ask}) = A\vec{s}.$$

To obtain the first ℓ results, compute $A[:\ell] \times \vec{s}$ (which is sufficient to compute $\vec{b} - A\vec{s}$, since $\vec{b}$ only has ℓ elements).

To further optimize the matrix multiplication $A \cdot \vec{s}$, instead of directly encrypting $\vec{s}$, UnifOMR encodes each coefficient of $\text{sk} := \sum_{i \in [n]} \text{sk}_i X^i$ as a constant polynomial: $s_i(X) := \text{sk}_i = \vec{s}[i]$, which is then encrypted under BFV: $\text{ct}_{s_i} \leftarrow \text{BFV.Enc}(\text{pp}_{\text{BFV}}, \text{pk}_{\text{BFV}}, s_i(X))$.

Hence, to (partially) decrypt the first bit of a clue $(a, \vec{b}) \in \mathcal{R}_t \times \mathbb{Z}_t^\ell$ (encrypting ℓ bits), the detector generates A as above and computes $b[0] - \sum_{i \in [n]} A[0][i] \times \text{ct}_{s_i}$. By leveraging the SIMD property of BFV (Section 4.5), D independent clues can be processed at the same time, where D is the ring dimension: Encode $A_1[0][i], \dots, A_D[0][i]$ for D different clues into the D “slots” of a plaintext ring element and then perform the multiplication. This procedure is performed for all $A[j], j \in [\ell]$ to (partially) decrypt for all ℓ bits.

To complete the RLWEenc decryption, a range check is performed afterwards, which results in a deep and expensive homomorphic circuit. Fortunately, this step can be done by the recipients in plaintext instead of being computed by the detector homomorphically. Thus, the detector only needs to perform the light-weight partial decryption. More formally, for a message, the detector performs the partial decryption via BFV, which results in (a BFV encryption of) $\vec{b}[j] - \text{ask}$ for $j \in [\ell]$. The recipient (decrypts the BFV ciphertext and) checks whether the resulting value

⁶This is achieved by setting $q = t$ for RLWEenc ciphertext modulus q .

of $\vec{b}[j] - ask \in [-r, r]$, for r being the error bound parameter of RLWEenc (see Definition 4.6), satisfying $((2r + 1)/t)^\ell < \epsilon_p$ for false positive rate ϵ_p . If so, this bit is decrypted to 0, and 1 otherwise.

If the final result of the i -th message is 0^ℓ , then the i -th message is pertinent. Otherwise, by RLWE, if $ct = (a, \vec{b}) \in \mathcal{R}_t \times \mathbb{Z}_t^\ell$ is encrypted under sk' , computing $\vec{b} - ask$ results in a vector in $\mathbb{Z}_t^\ell$ computationally indistinguishable from a uniform random vector, which is then decrypted into 0^ℓ with probability ϵ_p .

Savings from the strawman solution. One may wonder about the difference between this solution and the strawman solution, where all the clue ciphertexts are directly sent back to the recipient, and let the recipient perform the full decryption directly. To summarize, there are three major advantages for our construction:

- The communication cost for the strawman solution is $|c|$ per message, which can concretely be hundreds of bytes or even kilobytes as in [58, 59, 60, 51] (and UnifOMR). In contrast, UnifOMR only requires ℓ of $\mathbb{Z}_Q$ elements per message (where Q is the ciphertext modulus of BFV, being at most a few bytes, to support the linear operations needed).
- The recipient computation cost for the strawman solution is one ciphertext decryption per message. In contrast, UnifOMR only requires one BFV decryption per D messages, for ring dimension D .
- Lastly, UnifOMR allows additional optimizations to further reduce the communication cost, which will be detailed later.

The above arguments directly apply if the clue generation uses other widely used PKE schemes like RSA [82] or Discrete-Log-based solution (e.g., Diffie-Hellman Key Exchange [32] plus a symmetric key ciphertext, which acts as a PKE). We show the concrete advantage in Section 7.

BatchPIR for retrieval. After obtaining the indices of the pertinent messages, the recipients can use the batchPIR protocol (Definition 4.3) to retrieve the pertinent messages. This process is straightforward since batchPIR is used as a closed-box (which is also discussed or used in prior works [58, 47]), so we omit the details and include the formal description in Algorithm 1. After the recipient obtains the answer from batchPIR, the OMR retrieval procedure is completed.⁷

6.2 Full Construction with Reduced Communication

This section discusses how to further optimize the communication cost of the scheme.

Merging the errors. Notice that both RLWEenc and BFV generate RLWE ciphertexts that contain errors. Thus, if the two errors are merged, the communication cost can be further reduced (via modulus switching to be discussed later). However, with the naive realization of the partial decryption above, the errors of RLWEenc are encoded in *slots* while the errors of BFV are encoded in *coefficients*. With more syntactic rigor, the resulting BFV ciphertext $ct := (a, b) \in \mathcal{R}_Q^2$ satisfies $b - ask_{BFV} = e_{BFV} + \frac{Q}{t} \cdot y(X)$, where $y(X)$ encodes vector $\text{RingToVec}(e_{RLWEenc})$ satisfying that

⁷For clarity: UnifOMR uses batchPIR as a black box. Separately, we also prove that such (batch)PIR is implied by (SDKU) OMR. These two results are independent. In particular, the PIR used in the UnifOMR construction would not be, cyclically, the one given by the lower bound's reduction to UnifOMR.

Algorithm 1 UnifOMR: Practical Oblivious Message Retrieval

```

1: procedure UnifOMR.GenParam( $1^\lambda, \epsilon_p, \epsilon_n$ )
2:   Choose  $\text{pp}_{\text{BFV}} = (D, t, Q, \mathcal{D}_{\text{sk}}, Q')$  such that the error accumulation before modulus switching
   is smaller than  $Q/Q'$  (except for negligible probability) and  $D \geq n$  below; and choose  $\ell, q = t, \sigma$ 
   such that given modulus switching error (dependent on  $\mathcal{D}_{\text{sk}}$ )  $e_{\text{ModSW}}$ , let  $r' \leftarrow Q' \cdot r/t + e_{\text{ModSW}} + \lceil Q'/t \rceil$ ,
   it holds that  $(\frac{2\lceil r' \cdot t/Q' \rceil + Q'/t}{t})^\ell \leq \epsilon_p$ 
3:    $\text{pp}_{\text{PWW}} = (n, \ell, q, \sigma, r, \mathcal{D}) \leftarrow \text{RLWEenc.GenParam}(1^\lambda, \ell, q, \sigma, \epsilon_n)$ 
4:   return  $\text{pp} = (\text{pp}_{\text{BFV}}, \text{pp}_{\text{RLWEenc}}, \epsilon_p, \epsilon_n)$ 
5: procedure UnifOMR.KeyGen( $\text{pp}$ )
6:    $(\text{sk}_{\text{RLWEenc}}, \text{pk}_{\text{RLWEenc}}) \leftarrow \text{RLWEenc.KeyGen}(\text{pp}_{\text{RLWEenc}})$   $\triangleright$  Clue key
7:    $(\text{sk}_{\text{BFV}}, \text{pk}_{\text{BFV}}) \leftarrow \text{BFV.KeyGen}(\text{pp}_{\text{BFV}})$   $\triangleright$  Detection key
8:    $\text{ct}_{s_w} \leftarrow \text{BFV.Enc}(\text{pk}_{\text{BFV}}, \text{sk}_{\text{RLWEenc}}[w])$  for  $w \in [n]$   $\triangleright$  Detection key
9:   return  $(\text{sk} = (\text{sk}_{\text{BFV}}), \text{pk} = (\text{pk}_{\text{clue}} = \text{pk}_{\text{RLWEenc}}, \text{pk}_{\text{det}} = (\text{pk}_{\text{BFV}}, \text{ct}_{s_0}, \dots, \text{ct}_{s_{n-1}})))$ 
10: procedure UnifOMR.GenClue( $\text{pp}, \text{pk}_{\text{clue}}, x$ )
11:    $\vec{b} \leftarrow (0, \dots, 0) \in \mathbb{Z}_2^\ell$ 
12:    $c \leftarrow \text{RLWEenc.Enc}(\text{pk}_{\text{clue}}, \vec{b})$ 
13:   return  $c$   $\triangleright c \in \mathcal{R}_q \times \mathbb{Z}_q^\ell$ 
14: procedure UnifOMR.Retrieve0( $\text{pp}, \text{BB} = (x_i, c_i)_{i \in [N]}, \text{pk}_{\text{det}}, \bar{k}, \text{aux}_0 = \perp$ )
15:   for  $l \in [0, \lceil N/D \rceil - 1]$  do
16:     With  $c_{1+l \cdot D}, \dots, c_{D+l \cdot D}$  where  $c_i = (a_i, \vec{b}_i) \in \mathcal{R}_q \times \mathbb{Z}_q^\ell$ , generate  $A_i[\cdot]$  per  $A$  defined in
     Section 6.2.
17:     Generate  $b_j := \sum_{i \in [D]} \vec{b}_i[j] \cdot X^i$ 
18:     Generate  $a'_w := \sum_{i \in [D]} A[0][i] \cdot X^i$ 
19:      $\text{ct}_{l,j} \leftarrow b_j - \sum_{w \in [n]} a'_w \times \text{ct}_{s_w}$ 
20:   return  $\text{dig}_0 = (\text{ct}_{l,j})_{l \in [N/D], j \in [\ell]}$ 
21: procedure Decode0( $\text{pp}, \text{dig}, \text{sk}_{\text{det}}, \text{stR}_0 = \perp$ )
22:   Decrypt  $\text{ct}_{l,j} := (b_{l,j}, a_{l,j})$  to space  $\mathcal{R}_{Q'}$  (i.e.,  $b_{l,j} - a_{l,j} \text{sk}_{\text{BFV}} \in \mathcal{R}_{Q'}$ )
23:   Let  $m_{i,1}, \dots, m_{i,\ell} \in \mathbb{Z}_t^\ell$  denote the decrypted result for message  $i \in [N]$ . Check if they are
   all in range  $[-r' \cdot Q'/t, r' \cdot Q'/t]$ .
24:   If all the  $\ell \mathbb{Z}_t$  elements are in  $[-r, r]$ , add  $i$  to set  $S$ .
25:    $\text{pp}_{\text{PIR}} \leftarrow \text{batchPIR.GenParam}(1^\lambda, N)$  shared with the detector.
26:    $\triangleright$  Note that for batchPIR parameter generation, any parameters independent of  $N$  can be
   generated beforehand instead of generated here.
27:   If  $|S| \leq \bar{k}$ , append dummy values to  $S$  until  $|S| = \bar{k}$ . If  $|S| > \bar{k}$ , sample a random set  $S$  s.t.
    $|S| = \bar{k}$ .
28:    $(\text{qry}, \text{st}) \leftarrow \text{batchPIR.Query}(\text{pp}_{\text{PIR}}, S)$   $\triangleright$  We assume batchPIR is one-round and thereby omit
   the subscript for Query and Decode
29:   return  $\text{aux}_1 = \text{qry}, \text{stR}_1 = \text{st}$ 
30: procedure UnifOMR.Retrieve1( $\text{pp}, \text{BB} = (x_i, c_i)_{i \in [N]}, \text{pk}_{\text{det}}, \bar{k}, \text{aux}_1 = \text{qry}$ )
31:   return  $\text{dig}_i := \text{ans} \leftarrow \text{batchPIR.Ans}(\text{pp}_{\text{PIR}}, \text{qry}, \text{DB} = (x_i)_{i \in [N]})$ 
32: procedure UnifOMR.Decode1( $\text{pp}, \text{dig}_1, \text{sk}, \text{st}_1$ )
33:    $\cdot, \text{st}_1 \leftarrow \text{PIR.Decode}(\text{pp}_{\text{PIR}}, \text{ans}, \text{st}_0)$ 
34:   return  $\text{aux}_2 = \perp, \text{stR}_2 = \text{st}_1$ 

```

$e_{\text{RLWE}_{\text{Enc}}}[i]$ is small if the i -th message is pertinent, and random over $\mathbb{Z}_t$ otherwise. The goal is to obtain $\text{ct}' := (a', b') \in \mathcal{R}_Q^2$ such that $b' - a' \text{sk}_{\text{BFV}} = e'_{\text{BFV}} + \frac{Q}{t} \cdot e_{\text{RLWE}_{\text{Enc}}}$ directly.

One potential solution is to use the SlotToCoeff procedure (Appendix B) to homomorphically transform $y(X)$ into $e_{\text{RLWE}_{\text{Enc}}}$. This procedure requires a matrix multiplication, which can be merged with the matrix multiplication for partial decryption without any extra cost.

However, instead of taking this approach (which is potentially more general as discussed in Remark 6.3), we adopt a conceptually simpler idea. Observe that ct_{s_i} encrypts a constant polynomial. Multiplying a constant polynomial by a regular ring element is equivalent to multiplying a scalar by a vector. Thus, instead of encoding $A[j][i]$ into slots, the detector directly encodes them into the coefficients. For simplicity, assume $\ell = 1$ (i.e., only 1-bit to-be-decrypted). Let $p_i(X) := \sum_{j \in [D]} A_j[0][i] X^j$, and compute $\text{ct}' \leftarrow \sum_{i \in [n]} p_i(x) \cdot \text{ct}_{s_i}$. The result encrypts a polynomial whose j -th coefficient encodes $A_j[0] \times \vec{s}$. After obtaining ct' , the remaining task is to reduce the size of the ciphertext. This can be done by modulus switching ModSW: $\text{ct}'' \leftarrow \lfloor Q'/Q \cdot \text{ct}' \rfloor$ where $Q' \ll Q$ is another ciphertext modulus.

After ModSW, the resulting ciphertext $\text{ct}'' := (a'', b'') \in \mathcal{R}_{Q'}^2$ satisfies $b'' - a'' \text{sk}_{\text{BFV}} = e''_{\text{ModSW}} + \frac{Q'}{Q} \cdot e''_{\text{BFV}} + \frac{Q'}{t} \cdot e'_{\text{RLWE}_{\text{Enc}}}$, where $e'_{\text{RLWE}_{\text{Enc}}}[i]$ is small if the i -th message is pertinent, and random over $\mathbb{Z}_t$ if impertinent.

For example, one could set $t = Q'$.⁸ Then, by correctness of BFV, $e''_{\text{BFV}} \ll t/Q = Q'/Q$, thus approaches 0. The rest of the error is $e''_{\text{ModSW}} + e'_{\text{RLWE}_{\text{Enc}}}$: we have $e''_{\text{ModSW}} \leq h_{\text{BFV}}/2$, where h_{BFV} is the hamming weight of the BFV secret key (assuming ternary or binary secret key). On the other hand, $e'_{\text{RLWE}_{\text{Enc}}} = \Theta(h_{\text{RLWE}_{\text{Enc}}})$ (where $h_{\text{RLWE}_{\text{Enc}}}$ is the hamming weight of the RLWE_{Enc} secret key), being the error from public key encryption of RLWE_{Enc}. Thus, assuming $h_{\text{BFV}} = O(h_{\text{RLWE}_{\text{Enc}}})$ (achievable since both can be set as linear in the security parameter λ), the resulting error remains $O(h_{\text{RLWE}_{\text{Enc}}})$, the same as $|e_{\text{RLWE}_{\text{Enc}}}|$, while the ciphertext size is reduced by a factor of $\log(Q/t)$, which is significant in practice.

This procedure trivially extends to $\ell > 1$. And this concludes our UnifOMR construction, formalized in Algorithm 1.

Theorem 6.1. *UnifOMR in Algorithm 1 is an OMR construction (Definition 3.1) for any $\lambda > 0, 0 < \epsilon_p, \epsilon_n < 1$, assuming the hardness of RLWE, the correctness of BFV and batchPIR (being a one-round batch PIR). Additionally, UnifOMR is strong detection-key-unlinkable (Definition 3.5).*

Proof. • (Completeness) In line 19, the scheme obtains ciphertexts $\text{ct}_{l,j}$ for $l \in [[N/D]], j \in [\ell]$. Let $\text{ct} = (a, b) := \text{ct}_{0,0}$ (which applies to other l, j as well). By construction, we have $b - a \text{sk}_{\text{BFV}} = e_{\text{BFV}}(X) + e_{\text{RLWE}_{\text{Enc}}}(X)$ where $e_{\text{RLWE}_{\text{Enc}}}(X) = \sum_i e_{\text{RLWE}_{\text{Enc}},i}[0] \cdot X^i$ for $e_{\text{RLWE}_{\text{Enc}},i}$ denoting the error for clue i under sk . If the i -th coefficient corresponds to a pertinent message, $e_{\text{RLWE}_{\text{Enc}}}[i] \leq r$ (except for ϵ_n probability as guaranteed by line 3). Otherwise, $e_{\text{RLWE}_{\text{Enc}}}$ is uniform over $\mathbb{Z}_t$. Then, after modulus switching, the resulting $\text{ct}' := (a', b') \in \mathcal{R}_{Q'}$ satisfies $b' - a' \text{sk}_{\text{BFV}} = \lceil Q' \cdot e_{\text{BFV}}(X)/Q \rceil + \lceil \frac{Q'}{t} \cdot e_{\text{RLWE}_{\text{Enc}}}(X) \rceil + e_{\text{ModSW}}(X)$. Now, if the i -th coefficient corresponds to a pertinent message, the total error is bounded by $\lceil Q' \cdot e_{\text{BFV}}/Q \rceil + \lceil \frac{Q'}{t} \cdot r \rceil + e_{\text{ModSW}}$. When further scaled to t after decrypting the BFV ciphertext, we obtain $\lceil t \cdot e_{\text{BFV}}/Q \rceil + r + Q' \cdot e_{\text{ModSW}}/t \leq r + Q' \cdot e_{\text{ModSW}}/t + Q'/t \leq r'$ (since $e_{\text{BFV}} \leq Q/t$ by the correctness of BFV) by line 2.

⁸ Q' can be set differently as well, resulting in a different ciphertext-size and error tradeoff.

Therefore, except for negligible probability, the recipient obtains the indices that are pertinent. Then, by the correctness of **batchPIR**, the recipient obtains the correct payloads. Thus, the completeness of **UnifOMR** holds.

- (Soundness) As mentioned above, e_{RLWEenc} is uniform over $\mathbb{Z}_t$ for impertinent messages. Thus, in line 22, we obtain a value in $v \in \mathbb{Z}_{Q'}$, where if we compute $v \cdot t / Q' \in \mathbb{R}$, the integer value is uniform in $[0, t]$. Thus, the probability that the resulting element is in $[-r', r']$ is bounded by $(2\lceil r' \cdot t / Q' \rceil + 1) / t$. All the ℓ elements fall in this range with independent probability, thereby $= (\frac{2\lceil r' \cdot t / Q' \rceil + 1}{t})^\ell \leq \epsilon_p$ as guaranteed by line 2.
- (Privacy) Privacy is straightforward. We argue with a hybrid argument.
 - hyb_0 : same as our algorithm.
 - hyb_1 : replace the index set S with a randomly sampled index set S' with size $\bar{k}$ for the **batchPIR** query.
 - hyb_2 : replace the detection key with uniformly random ring elements.
 - hyb_3 : replace the clue key with uniformly random ring elements.

hyb_0 and hyb_1 are indistinguishable by the privacy guarantee of **batchPIR** (note that in our original scheme, we always query with some set S with size $\bar{k}$ *regardless* of the reply from the first round). hyb_1 and hyb_2 are indistinguishable by the underlying RLWE assumption (since the encrypted sk_{clue} is sampled independently from $\text{sk}_{\text{det}} = \text{sk}_{\text{BFV}}$). hyb_2 and hyb_3 are indistinguishable again by the underlying RLWE assumption. Lastly, hyb_3 is trivially secure again the Fig. 2 game. Thus, **UnifOMR** satisfies the privacy guarantee.

- (Strong-detection-key-unlinkability) Lastly, strong-detection-key-unlinkability is similar to privacy. Even obtaining sk_{clue} , the detection key and **batchPIR** queries are still indistinguishable from uniform random (since there security is dependent on sk_{BFV} , which is sampled independently from sk_{clue}).

□

Remark 6.2. In [58], a primitive call Oblivious Message Detection (OMD) is also defined. The difference is that OMD only requires the recipients to obtain the pertinent indices instead of the real payloads. One can alternatively view OMD as OMR, but all the payloads are just indices instead of arbitrary bit strings.

UnifOMR can alternatively be viewed as a realization of OMD by using linear homomorphic encryption and then transformed into OMR by utilizing **batchPIR**. We call the OMD variant of our scheme **UnifOMD**.

Remark 6.3. As mentioned, the method of using **SlotToCoeff** is more general, since in some cases, one may want the detection key to only include a single BFV ciphertext encrypting sk directly (lifted from the ring used for **RLWEenc** to the ring used for BFV) to reduce the detection key size. In this case, the coefficient embedding does not work. However, similarly, we can still use the slot embedding and then use the **SlotToCoeff** trick above to move the errors to the coefficients. Additionally, as mentioned, **SlotToCoeff** is also a matrix multiplication and can therefore be merged with $A \times \vec{s}$. I.e., **SlotToCoeff** means $U \times A \times \vec{s}$ for some matrix U , and the detector could compute $U \times A$ before multiplying $\vec{s}$, which is the encrypted value. Such case is very application dependent and thereby we leave it for future works to explore its use cases.

6.3 Additional Optimizations

This section introduces some additional optimizations that are orthogonal to the scheme itself.

Balancing communication by tuning false positive rate. One may notice that the detection (partial decryption) portion of UnifOMR has a communication cost of $O(N \cdot \log(1/\epsilon_p))$, while the PIR portion of UnifOMR has a communication cost of $O(k + N \cdot \epsilon_p)$ ⁹. Thus, one could set a larger ϵ_p to slightly increase the total number of messages to be fetched using batchPIR to reduce the total communication cost. On the other hand, since batchPIR communication cost is also dependent on the payload size, which could be megabytes or larger for a pictorial or video message, the batchPIR component may dominate the communication cost. In this case, ϵ_p should be set to be as low as possible.

Note that the change of false positive rate, in general, does *not* cause any issue as discussed in [58]. Specifically, the payloads should also be encrypted. Thus, the clients should be able to distinguish true positives from false positives easily. Similar situations have been encountered by PIR as well: PIR queries usually contain more than the queried entries themselves to ensure efficiency (e.g., see [68, 43]). Thus, we believe adjusting the false positive rate for better efficiency is preferred in most cases.

NTT preprocessing. Naively performing a ring element multiplication takes $O(D^2)$ time where D is the ring dimension. Thus, instead, an implementation usually performs a Number Theoretic transformation (NTT) over the two to-be-multiplied ring elements, which takes $D \log(D)/2$ field-element multiplications per NTT, multiply the NTT form ring elements, which takes D field-element multiplications, performs element-wise multiplications between the two NTT form ring elements, which takes D field-element multiplications, and then performs an inverse NTT (INTT) back with the same cost as NTT. This process instead takes $O(D \log(D))$ time.

For UnifOMR, this can be further improved. Specifically, for detection keys being $\text{Enc}(\text{sk}_{\text{clue}}[i])$, the keys can be directly uploaded as in NTT form. Then, for each b_j, a'_w (line 19), the server first performs an NTT transformation on them (which can be used over all the recipients this detector is assisting) and then performs the element-wise multiplication in its NTT form. Lastly, it performs the summation in the NTT form first before transforming it back using INTT. In this case, the total cost is only $nD + D \log(D)/2$ multiplications per D messages (ignoring the NTT that can be used for all clients), where n is the secret key dimension for the clue.

7 Evaluation

We present our experiment benchmarks in this section by starting with parameter selection.

7.1 Parameter Selection

The parameters needed for UnifOMR are:

- n (RLWEenc secret key dimension)¹⁰,
- q (RLWEenc ciphertext modulus),

⁹This is dependent on the batchPIR scheme, but normally linear in k .

¹⁰If n is not a power of two, we find the smallest power of two $n' > n$ and pad n with zeros, so we can work over the cyclotomic ring we need. Then, we skip these zeros during homomorphic decryption to save computation costs.

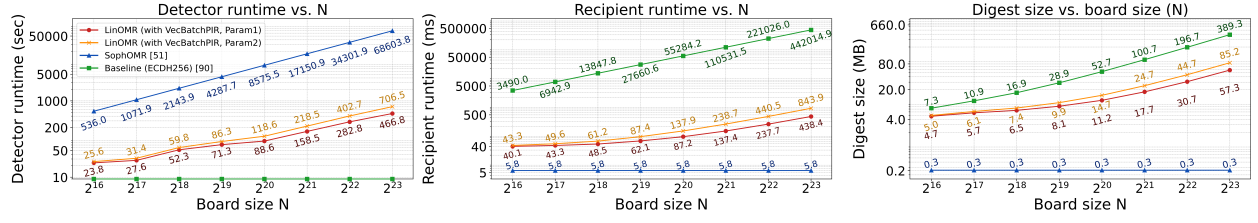


Figure 7: Efficiency for varying number of messages N , with fixed $k = 500$, $P = 612$ bytes, log-log scale. The baseline has no detector computation in the leftmost subplot; the line is added for illustrative purposes in the legend, which applies to all subplots.

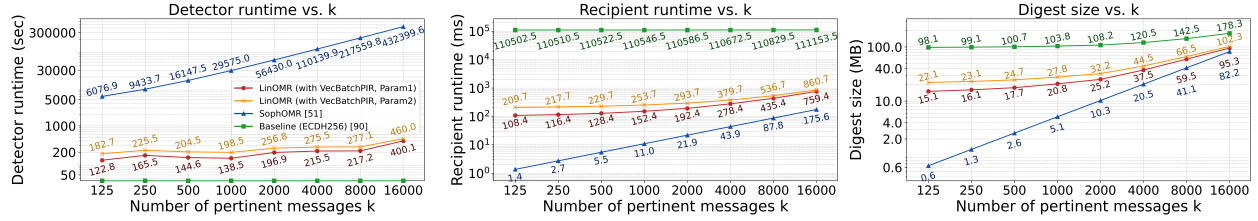


Figure 8: Efficiency for varying (cap on the) number of pertinent messages k , with fixed $N = 2^{21}$, $P = 612$ bytes, log-log scale. The baseline has no detector computation in the leftmost subplot; the line is added for illustrative purposes in the legend, which applies to all subplots.

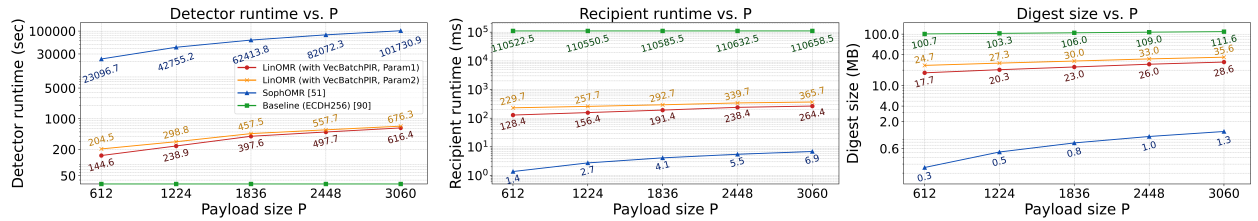


Figure 9: Efficiency for varying payload size P in bytes, with fixed $N = 2^{21}$, $k = 500$, log-log scale. The baseline has no detector computation in the leftmost subplot; the line is added for illustrative purposes in the legend, which applies to all subplots.

	n	q	h_{RLWEenc}	r	ℓ	σ	D	Q	h_{BFV}	Q'	σ_{BFV}	r'	ϵ_n	ϵ_p	Security
Param1	900	4169729	80	48	1	0.6	2048	$\approx 2^{60}$	400	67104769	3.2	$61 \cdot 16 + 16$	2^{-30}	2^{-15}	110-bit
Param2	1024	1032193	80	84	2	0.5	4096	$\approx 2^{60}$	128	$= q$	3.2	149	2^{-128}	2^{-22}	$> 128\text{-bit}$

Table 1: Parameter choices for UnifOMR. n : RLWEenc secret key dimension; q : RLWEenc ciphertext modulus; h_{RLWEenc} : hamming weight of RLWEenc secret key; r : RLWEenc error range bound; ℓ : number of bits encrypted by RLWEenc ciphertexts; σ : RLWEenc error standard deviation; D : BFV ring dimension; Q : BFV initial ciphertext modulus; h_{BFV} : hamming weight of BFV secret key; Q' : BFV ciphertext modulus after modulus switching; σ_{BFV} : BFV error standard deviation; r' : final error range bound after error merge via modulus switching; ϵ_n : false negative rate; ϵ_p : false positive rate; λ : computational security parameter.

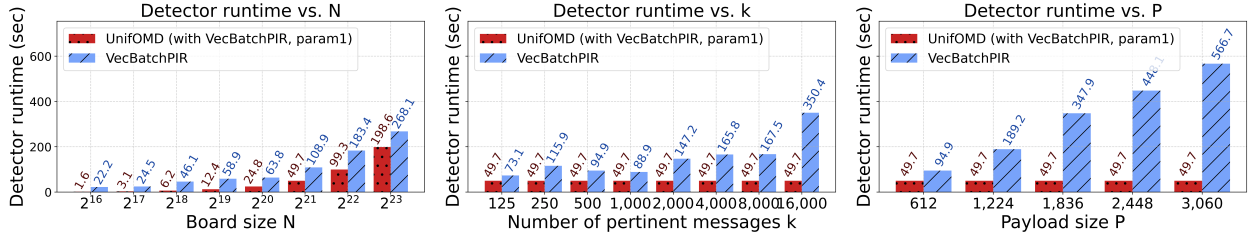


Figure 10: Detector runtime breakdown between detection (via UnifOMD, Param1) and retrieval (via VecBatchPIR [73]) given different parameters. Unless explicitly varied on the x-axis, we fix the parameters to $N = 2^{21}$, $P = 612$ bytes, and $k = 500$. From left to right, the plots are linear-log scaled, linear-log scaled, and linear-linear scaled, respectively.

- h_{RLWEenc} (hamming weight of RLWEenc secret key),
- r (RLWEenc error range bound),
- ℓ (number of bits encrypted by RLWEenc ciphertexts),
- σ_{RLWEenc} (RLWEenc error standard deviation),
- D (BFV ring dimension),
- Q (BFV initial ciphertext modulus),
- h_{BFV} (hamming weight of BFV secret key),
- Q' (BFV ciphertext modulus after modulus switching),
- σ_{BFV} (BFV error standard deviation),
- r' (final error range bound after error merge via modulus switching),
- ϵ_n (false positive rate),
- ϵ_p (false negative rate),
- λ (computational security parameter).

False negative rate ϵ_n . To choose parameters given ϵ_n , the main consideration is Q' and r, r' . Essentially, we want to make sure that after merging, the resulting error stays in range $[-r', r']$ with at least $1 - \epsilon_n - \text{negl}(\lambda)$ probability. As calculated in the proof of Theorem 6.1, the error bound is $r + Q' \cdot e_{\text{ModSW}}/t + 1$. Then, r is a Gaussian error with standard deviation $\sqrt{2 \cdot h_{\text{RLWEenc}} + 1} \cdot \sigma$ (see [60] for details), and e_{ModSW} is simply bounded by $h_{\text{BFV}}/2$ (since the rounding error of each coefficient is at most 0.5). Thus, we bound the false negative probability using this.

False positive rate ϵ_p . Given, r' , we have $\epsilon_p \leq ((2\lfloor t \cdot r'/Q' \rfloor + 1)/t)^\ell$ as calculated in the proof of Theorem 6.1. Thus, given r', Q', t set above according to ϵ_n , we can set ℓ accordingly given ϵ_p .

BFV noise. Lastly, we want to guarantee that BFV is correctly computed, so the noise before the merging step should be smaller than $\lfloor \log(Q) \rfloor / \lceil \log(t) \rceil - 1$. We can analyze the error as follows: The circuit consists of an addition of $n + 1$ additions, and one level of plaintext multiplication. Given an initial ciphertext with error standard deviation σ_{BFV} , one plaintext multiplication grows the error by a multiplicative factor of $\log(t) \cdot \sqrt{D}$. Then, the additions introduce a multiplicative factor of $\sqrt{n + 1}$. Thus, the final error standard deviation becomes $\log(t) \cdot \sqrt{D} \cdot \sqrt{n + 1} \cdot \sigma_{\text{BFV}}$. We simply need to bound this error to be within $\lfloor \log(Q) \rfloor / \lceil \log(t) \rceil - 1$ bits (except for negligible probability).

Parameter set 1. We first choose a parameter set to minimize the computation and communication of UnifOMD, while guaranteeing BFV correctness and $\epsilon_n = 2^{-30}$ as in prior works ([58, 59, 60, 51]). It sacrifices ϵ_p by a bit (about 2^{-15} instead of 2^{-20} as in prior works). Security-wise, this parameter set provides 110-bit computational security as [60], per LWE-estimator [2]. We believe that this parameter set is sufficient for most use cases and use it as our main result.¹¹

Parameter set 2. We then choose a parameter set which provides $\epsilon_n = 2^{-128}$ and 128-bit computational security (per LWE-estimator [2]), and thus is sufficient for essentially *any* real-world applications. The ϵ_p is also reduced to 2^{-22} , which is not as small as ϵ_n or λ , since as discussed in Section 6.3, ϵ_p is only relevant for efficiency.

7.2 Comparison and Benchmark

We implemented UnifOMR in C++, available at [77], based on the OMR library [77], which is based on SEAL [70] and PALISADE (OpenFHE) [78, 8]. We first compare our Oblivious Message Detection (OMD) component, denoted as UnifOMD (see Remark 6.2 for more discussion), with the state-of-the-art construction SophOMD, [51], and also an additional baseline, the strawman discussed in Section 6.1, instantiated with Elliptic Curve Diffie-Hellman (ECDH) key exchange [32, 71, 49] plus AES [75].¹² Then, we compare our OMR construction, UnifOMR (instantiated with VecBatchPIR [73] as the batchPIR construction¹³), with SophOMR.

We run all the benchmarks on a Google Compute Cloud n2-standard-16 instance with 16GB RAM for a fair comparison for the representative costs below. For additional parameters, a larger memory is needed to store the database. We use n2-standard-128 with 512GB RAM for benchmarking the underlying VecBatchPIR scheme [73]. All of the following benchmarks for our constructions have been run for ≥ 10 times with a standard deviation of $< 5\%$, and we report the average.

¹¹See Footnote 10 for how $n = 900$, not a power-of-two, is used.

¹²We consider instantiating it using ECDH since other mainstream solutions have worse efficiency, like RSA [82] or Crystals-Kyber [84]. We include a comparison in Appendix D.1.

¹³See Appendix D.3 for discussion on why we chose VecBatchPIR.

	Detector runtime (sec)	Digest size (KB)	Recipient runtime (ms)	Detection key (MB)	Clue key (bytes)	Clue (bytes)	Rounds of communication
Baseline (ECDH [90])	N/A	24576	27623	N/A	32	48	1
SophOMD (OMD) [51]	743	132	14	114	2576	2565	
UnifOMD (Param1)	12	3328	25	13	2491	2477	
UnifOMD (Param2)	27	5120	51	30	2576	2565	
Baseline (ECDH [90]) + batchPIR	14	24576	27623	18	32	48	2
SophOMR (OMR) [51]	1258	263	6	142	2576	2565	1
UnifOMR (Param1)	25	4163	28	31	2491	2477	2
UnifOMR (Param2)	41	5955	54	48	2576	2565	

Table 2: Efficiency comparison given parameters: total number of messages $N = 2^{19}$, (the cap on the number of pertinent messages $k = \bar{k} = 50$, payload size $P = 612$ bytes, as in [58, 59, 60, 51]. UnifOMD is the detection component (i.e., recipients receive only pertinent indices, not payloads) of UnifOMR and SophOMD is its counterpart of SophOMR.

Note that for SophOMR [51], the provided open-sourced implementation only supports up to $N = 2^{20}$ (and errors occur with $N \geq 2^{20}$), so we benchmark up to $N = 2^{20}$. However, the detector procedure of SophOMR for $N > 2^{16}$ is *precisely* running the *same* algorithm for $N/2^{16}$ times.¹⁴ Hence, we estimate the runtime for larger N given the benchmarks for $N \leq 2^{20}$. We benchmarked for $N = 2^{16}$ to 2^{20} to verify that the behavior is indeed as expected.

Representative costs. As shown in Table 2, we first compare our OMR component with the baseline solution (ECDH + SKE; see Appendix D.1), and SophOMD. Compared to the baseline UnifOMD (Param1) has $> 1000\times$ smaller recipient runtime and $\sim 7\times$ smaller digest size. This greatly reduces recipients’ resource burden. When compared to SophOMD, UnifOMD (Param1) has $> 60\times$ smaller detector runtime, which, as discussed, is the main performance bottleneck of OMR/OMD. Furthermore, UnifOMD has about $9\times$ smaller detection key size compared to SophOMD.

A similar conclusion (but slightly smaller advantage) can be drawn for our OMR construction UnifOMR. For Param2, our efficiency is $< 2\times$ worse than our Param1 for all metrics. However, we emphasize that Param1 should be sufficient for most applications in practice, since it guarantees similar security and false negative rate ϵ_n as prior works [58, 59, 60].

Efficiency with respect to N . Fig. 7 shows the detector runtime, recipient runtime, and communication costs with varying $N = 2^{16}$ to 2^{23} (again, for SophOMR with $N > 2^{20}$, we scale it as discussed above) and fixed $k = 500$, payload size $P = 612$ bytes. The detector runtime of UnifOMR is $> 100\times$ faster than SophOMR for large N , while both scale linearly. Our client runtime increases with N as well, but stays under 1 second even for $N = 2^{23}$, compared to > 400 seconds for the baseline.

In terms of digest size, our scheme scales linearly while SophOMR stays constant. However, even for $N = 2^{23}$, the digest size is only $\sim 57\text{MB}$ for Param1. Given a 13MB/sec bandwidth (which is below the average world download speed on mobile phones [1]), it takes only < 5 seconds to download the digest. Thus, we believe such a tradeoff for detector time is greatly preferred in most applications.

Efficiency with respect to k . Fig. 8 shows the detector runtime, client runtime, and communication costs with varying k from 125 to 16000 and fixed $N = 2^{21}$ and payload size $P = 612$

¹⁴Technically, for larger N , additional noise analysis might be necessary for the underlying BFV/BGV, so the runtime for prior works may be slightly worse. However, we ignore this potential overhead.

bytes.¹⁵ Our detector runtime fluctuates with the underlying batchPIR runtime. As k increases, our detector runtime can be more than *three* orders of magnitude smaller than **SophOMR**, and our communication costs become very similar to **SophOMR** as well. At the same time, while our recipient runtime remains larger than **SophOMR**, it retains less than 1 second, $> 1000\times$ smaller than the baseline solution.

Efficiency with respect to payload size. As in prior single-server OMR work, we focus on the payload size $P = 612$ bytes, corresponding to Zcash Sapling transactions [45] (see [58]). However, this size may vary a lot by application. Thus, we demonstrate how the performance of **UnifOMR** changes with respect to payload size. Fig. 9 shows the detector runtime, client runtime, and communication costs with varying payload size from 612 to 3060 and fixed $N = 2^{21}$, $k = 500$. We achieve $> 100\times$ speedup over **SophOMR** regarding the detector runtime with recipient runtime under 0.4 seconds and ~ 30 MB digest.

Cost breakdown. Fig. 10 illustrates the detector runtime breakdown of **UnifOMR**, which boils down to **UnifOMD** and **VecBatchPIR**. By fixing the payload size at $P = 612$ bytes and the number of pertinent messages at $k = 500$, the underlying PIR component accounts for roughly two-thirds of the total detector runtime. As either P or k increases (with the board size fixed at $N = 2^{21}$), the share of **VecBatchPIR** progressively comes to dominate the overall runtime, rising to $\sim 90\%$ for large parameters. We include additional efficiency breakdown and that of the parameter set 2 in Appendix D.2.

Latency. A very recent concurrent work **InstantOMR** [54] proposed the notion of “latency” for OMR, which is the total server time for a single message (or a small batch of messages). **InstantOMR** achieves low latency (< 0.1 second for a single message) at the cost of having higher throughput (overall detector runtime). Specifically, for 2^{16} messages (or all parameters we benchmark), the detector runtime of **InstantOMR** is $> 10\times$ slower than **SophOMR**, thereby $> 250\times$ slower than our **UnifOMR**. Even in terms of latency, **UnifOMR** is only < 2 seconds. While this is not as low as **InstantOMR**, our latency is $> 26\times$ lower than **SophOMR**, and we believe such a low latency is sufficient for most applications. We discuss these and compare **UnifOMR** with **InstantOMR** in more detail in Appendix D.4.

8 Conclusion

In this work, we addressed two central open questions for Oblivious Message Retrieval (OMR).¹⁶ First, we established a formal relationship between OMR and Private Information Retrieval (PIR), showing that OMR (with strong detection-key-unlinkability) is at least as hard as PIR as a cryptographic primitive. We show that such OMR constructions can be used to build PIR with nearly no overhead. This result not only clarifies the conceptual boundaries between the two but also implies inherent efficiency constraints for OMR schemes, both asymptotically and concretely.

Second, we show that concretely we can indeed have an OMR construction that achieves efficiency close to the batchPIR construction we use as a blackbox. Specifically, we proposed a new OMR construction, **UnifOMR**, that significantly improves detector runtime, achieving $20\times$ to $1080\times$ speedup in terms of detector runtime over the state-of-the-art **SophOMR** for parameters benchmarked above, which is the main performance bottleneck of OMR, as recognized in prior

¹⁵Again, for **SophOMR**, we use $N = 2^{20}$ runtime multiplied by two instead. However, as mentioned, this is accurate.

¹⁶See also Appendix E for more open questions.

works. Furthermore, the underlying batchPIR construction takes 50% to 92% of the total detector runtime.

Suitable applications. UnifOMR achieves such speedup at the cost of having the digest size being larger than prior works. However, as shown in Section 7, for representative parameters tested in prior works, we reduce the detector runtime from > 1250 seconds to only 25 seconds, at the cost of enlarging the digest size by $\sim 4\text{MB}$, which takes 0.3 seconds to download given 13MB/sec bandwidth (below the average world download speed on mobile phones [1]). For the extensive parameters we have tested in Section 7, our digest size is kept in $\lesssim 100\text{MB}$, which takes < 8 seconds to download, while saving the detector runtime in the order of *hours or even days* for such a digest size cost. Thus, we believe that this tradeoff is preferable in most real-world applications.

Acknowledgements

We are grateful to the anonymous reviewers for their insightful suggestions. We thank Jingyu Li for answering questions regarding PIRANA [56]. Zeyu Liu is supported by Yale CADMY.

References

- [1] Speedtest global index, July 2025. <https://www.speedtest.net/global-index>.
- [2] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, pages 169–203, 2015.
- [3] Asra Ali, Tancrede Lepoint, Sarvar Patel, Mariana Raykova, Phillipp Schoppmann, Karn Seth, and Kevin Yeo. Communication–Computation trade-offs in PIR. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1811–1828. USENIX Association, August 2021.
- [4] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 2018.
- [5] Diego F. Aranha, Anamaria Costache, Antonio Guimarães, and Eduardo Soria-Vazquez. HELIOPOLIS: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part V*, volume 15488 of *LNCS*, pages 302–334. Springer, Singapore, December 2024.
- [6] Thomas Attema, Pedro Capitão, and Lisa Kohl. On homomorphic secret sharing from polynomial-modulus LWE. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part II*, volume 13941 of *LNCS*, pages 3–32. Springer, Cham, May 2023.
- [7] Adam Back, Ulf Möller, and Anton Stiglic. Traffic analysis attacks and trade-offs in anonymity providing systems. In *Proceedings of the 4th International Workshop on Information Hiding, IHW ’01*, page 245–257, Berlin, Heidelberg, 2001. Springer-Verlag.
- [8] Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Sponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. OpenFHE: Open-source fully homomorphic encryption library. *Cryptology ePrint Archive*, Paper 2022/915; WAHC’22, 2022.
- [9] Gabrielle Beck, Julia Len, Ian Miers, and Matthew Green. Fuzzy message detection. *ACM CCS* 2021.
- [10] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers’ computation in private information retrieval: PIR with preprocessing. *Journal of Cryptology*, 17(2):125–151, March 2004.
- [11] Mihir Bellare, Alexandra Boldyreva, Anand Desai, and David Pointcheval. Key-privacy in public-key encryption. In *ASIACRYPT 2001*, LNCS, pages 566–582, December 2001.
- [12] Eli Ben Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE S&P*, pages 459–474, 2014.

- [13] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE Computer Society Press, May 2014.
- [14] Olivier Bernard, Marc Joye, Nigel P. Smart, and Michael Walter. Drifting towards better error probabilities in fully homomorphic encryption schemes. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 181–211. Springer, Cham, May 2025.
- [15] Alexander Bienstock, Sarvar Patel, Joon Young Seo, and Kevin Yeo. Batch PIR and labeled PSI with oblivious ciphertext compression. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.
- [16] Andrea Bittau, Úlfar Erlingsson, Petros Maniatis, Ilya Mironov, Ananth Raghunathan, David Lie, Mitch Rudominer, Ushasree Kode, Julien Tinnes, and Bernhard Seefeld. Prochlo: Strong privacy for analytics in the crowd. In *SOSP*, pages 441–459, 2017.
- [17] Grant Bosworth, Keewoo Lee, and Sunwoong Kim. Leveraging fpgas for homomorphic matrix-vector multiplication in oblivious message retrieval. In *Asilomar Conference on Signals, Systems, and Computers*, 2025.
- [18] Sean Rowe, Alessandro Chiesa, Matthew Green, Ian Miers, Pratyush Mishra, and Howard Wu. Zexe: Enabling decentralized private computation. In *2020 IEEE S&P (SP)*, pages 947–964, 2020.
- [19] Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 509–539. Springer, Berlin, Heidelberg, August 2016.
- [20] Elette Boyle, Lisa Kohl, and Peter Scholl. Homomorphic secret sharing from lattices without FHE. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 3–33. Springer, Cham, May 2019.
- [21] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In *CRYPTO 2012*, LNCS. Springer, August 2012.
- [22] Alexander Burton, Samir Jordan Menon, and David J. Wu. Respire: High-rate PIR for databases with small records. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 1463–1477. ACM Press, October 2024.
- [23] Marina Checri, Renaud Sirdey, Aymen Boudguiga, and Jean-Paul Bultel. On the practical CPA^D security of “exact” and threshold FHE schemes and libraries. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 3–33. Springer, Cham, August 2024.
- [24] Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. Attacks against the IND-CPA^D security of exact FHE schemes. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2505–2519. ACM Press, October 2024.

- [25] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 360–384. Springer, Cham, April / May 2018.
- [26] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *36th FOCS*, pages 41–50. IEEE Computer Society Press, October 1995.
- [27] Haotian Chu, Xiao Wang, and Yanxue Jia. Private signaling secure against actively corrupted servers. Cryptology ePrint Archive, Paper 2025/1056, 2025.
- [28] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An anonymous messaging system handling millions of users. In *2015 IEEE S&P*, pages 321–338, 2015.
- [29] Henry Corrigan-Gibbs, Alexandra Henzinger, Yael Tauman Kalai, and Vinod Vaikuntanathan. Somewhat homomorphic encryption from linear homomorphism and sparse LPN. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 3–33. Springer, Cham, May 2025.
- [30] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 44–75. Springer, Cham, May 2020.
- [31] Giovanni Di Crescenzo, Tal Malkin, and Rafail Ostrovsky. Single database private information retrieval implies oblivious transfer. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 122–138. Springer, Berlin, Heidelberg, May 2000.
- [32] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- [33] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012. <https://ia.cr/2012/144>.
- [34] Ben Fisch, Arthur Lazzaretti, Zeyu Liu, and Charalampos Papamanthou. ThorPIR: Single server PIR via homomorphic thorp shuffles. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 1448–1462. ACM Press, October 2024.
- [35] Mariana Gama, Emad Heydari Beni, Jiayi Kang, Jannik Spiessens, and Frederik Vercauteren. Blind zkSNARKs: for private proof delegation and verifiable computation over encrypted data. *CiC*, 2(3):10, 2025.
- [36] Sanjam Garg, Aarushi Goel, and Mingyuan Wang. How to prove statements obliviously? In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 449–487. Springer, Cham, August 2024.
- [37] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *ACM Symposium on Theory of Computing*, STOC ’09, page 169–178. ACM, 2009.
- [38] Yael Gertner, Sampath Kannan, Tal Malkin, Omer Reingold, and Mahesh Viswanathan. The relationship between public key encryption and oblivious transfer. In *41st FOCS*, pages 325–335. IEEE Computer Society Press, November 2000.

- [39] Ashrujit Ghoshal, Baitian Li, Yaohua Ma, Chenxin Dai, and Elaine Shi. Scalable multi-server private information retrieval. LNCS, pages 582–610. Springer, Cham, November 2025.
- [40] Tiantian Gong, Ryan Henry, Alexandros Psomas, and Aniket Kate. More is merrier: Relax the non-collusion assumption in multi-server PIR. In *2024 IEEE Symposium on Security and Privacy*, pages 4348–4366. IEEE Computer Society Press, May 2024.
- [41] Tiantian Gong, Aniket Kate, Hemanta K. Maji, and Hai H. Nguyen. Disincentivize collusion in verifiable secret sharing. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part V*, volume 15605 of LNCS, pages 34–64. Springer, Cham, May 2025.
- [42] Tiantian Gong and Zeyu Liu. Rational secret sharing with competition. *Financial Cryptography and Data Security 2025 (FC 25)*, 2025.
- [43] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023*, pages 3889–3905. USENIX Association, August 2023.
- [44] Alexandra Henzinger and Seyoon Ragavan. Two-server private information retrieval in sub-linear time and quasilinear space. *Eurocrypt 2026*, 2026.
- [45] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash Protocol Specification Version 2025.8.20. <https://zips.z.cash/protocol/protocol.pdf>, 2025.
- [46] Sashidhar Jakkamsetti, Zeyu Liu, and Varun Madathil. Scalable private signaling. *CSF 2025*, 2025. <https://eprint.iacr.org/2023/572>.
- [47] Yanxue Jia, Varun Madathil, and Aniket Kate. HomeRun: High-efficiency oblivious message retrieval, unrestricted. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2012–2026. ACM Press, October 2024.
- [48] Jaehyung Kim, Hanjun Li, Huijia Lin, and Zeyu Liu. SIMD HSS and aHMAC from interval encoding with application to one-bit-per-gate garbling. *Cryptology ePrint Archive*, Paper 2026/485, 2026.
- [49] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987.
- [50] Arthur Lazzaretti, Zeyu Liu, Ben Fisch, Peihan Miao, and Charalampos Papamanthou. Multi-server doubly efficient PIR in the classical model and beyond. LNCS, pages 611–641. Springer, Cham, November 2025.
- [51] Keewoo Lee and Yongdong Yeo. SophOMR: Improved oblivious message retrieval from SIMD-aware homomorphic compression. *USENIX Security 2026*, to appear, 2026.
- [52] Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of LNCS, pages 648–677. Springer, Cham, October 2021.

- [53] Baiyu Li, Daniele Micciancio, Mariana Raykova, and Mark Schultz. Hintless single-server private information retrieval. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IX*, volume 14928 of *LNCS*, pages 183–217. Springer, Cham, August 2024.
- [54] Hao-fei Liang, Zeyu Liu, Eran Tromer, Xiang Xie, and Yu Yu. InstantOMR: Oblivious message retrieval with low latency and optimal parallelizability. *USENIX Security 2026*, 2026.
- [55] Wei-Kai Lin, Ethan Mook, and Daniel Wichs. Doubly efficient private information retrieval and fully homomorphic RAM computation from ring LWE. In Barna Saha and Rocco A. Servedio, editors, *55th ACM STOC*, pages 595–608. ACM Press, June 2023.
- [56] Jian Liu, Jingyu Li, Di Wu, and Kui Ren. Pirana: Faster multi-query pir via constant-weight codes. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4315–4330. IEEE, 2024.
- [57] Zeyu Liu, Katerina Sotiraki, Eran Tromer, and Yunhao Wang. Snake-eye resistant pke from lwe for oblivious message retrieval and robust encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 126–156. Springer, 2025.
- [58] Zeyu Liu and Eran Tromer. Oblivious message retrieval. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, pages 753–783, Cham, 2022. Springer Nature Switzerland. Full version: Cryptology ePrint Archive 2021; internal citations follow the latter’s numbering.
- [59] Zeyu Liu, Eran Tromer, and Yunhao Wang. Group oblivious message retrieval. *S&P 2024*. Full version on eprint <https://ia.cr/2023/534>, 2023.
- [60] Zeyu Liu, Eran Tromer, and Yunhao Wang. Performr: Oblivious message retrieval with reduced communication and computation. *USENIX Security 2024*, 2024. Full version of this paper. Available on print: <https://eprint.iacr.org/2024/204>.
- [61] Zeyu Liu and Yunhao Wang. Amortized functional bootstrapping in less than 7ms, with $\tilde{O}(1)$ polynomial multiplications. *Asiacrypt 2023*. <https://eprint.iacr.org/2023/910>.
- [62] Zeyu Liu and Yunhao Wang. Relaxed functional bootstrapping: A new perspective on bgv/bfv bootstrapping. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 208–240. Springer, 2024.
- [63] Zeyu Liu, Yunhao Wang, and Ben Fisch. IND-CPA-D of relaxed functional bootstrapping: A new attack, A general fix, and A stronger model. In *ACM CCS 2025*, pages 2907–2921. ACM Press, November 2025.
- [64] Joshua Lund. Technology preview: Sealed sender for signal. <https://signal.org/blog/sealed-sender/>, Oct. 2018.
- [65] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. *J. ACM*, 2013.
- [66] Varun Madathil, Alessandra Scafuro, István András Seres, Omer Shlomovits, and Denis Var-lakov. Private signaling. *USENIX Security 2022*, 2022.

- [67] Nick Mathewson and Roger Dingledine. Practical traffic analysis: extending and resisting statistical disclosure. In *Proceedings of the 4th International Conference on Privacy Enhancing Technologies*, PET’04, page 17–34, Berlin, Heidelberg, 2004. Springer-Verlag.
- [68] Samir Jordan Menon and David J. Wu. SPIRAL: Fast, high-rate single-server PIR via FHE composition. In *2022 IEEE Symposium on Security and Privacy*, pages 930–947. IEEE Computer Society Press, May 2022.
- [69] Samir Jordan Menon and David J. Wu. YPIR: High-throughput single-server PIR with silent preprocessing. In Davide Balzarotti and Wenyan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.
- [70] Microsoft SEAL (release 3.6). <https://github.com/Microsoft/SEAL>, November 2020. Microsoft Research, Redmond, WA.
- [71] Victor S. Miller. Use of elliptic curves in cryptography. In Hugh C. Williams, editor, *CRYPTO’85*, volume 218 of *LNCS*, pages 417–426. Springer, Berlin, Heidelberg, August 1986.
- [72] Prateek Mittal, Ahmed Khurshid, Joshua Juen, Matthew Caesar, and Nikita Borisov. Stealthy traffic analysis of low-latency anonymous communication using throughput fingerprinting. In Yan Chen, George Danezis, and Vitaly Shmatikov, editors, *ACM CCS 2011*, pages 215–226. ACM Press, October 2011.
- [73] Muhammad Haris Mughees and Ling Ren. Vectorized batch private information retrieval. In *2023 IEEE Symposium on Security and Privacy*, pages 437–452. IEEE Computer Society Press, May 2023.
- [74] Milad Nasr, Amir Houmansadr, and Arya Mazumdar. Compressive traffic analysis: A new paradigm for scalable traffic analysis. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 2053–2069. ACM Press, October / November 2017.
- [75] National Institute of Standards and Technology (NIST). Fips 197: Advanced encryption standard (AES). Technical report, NIST, 2001. Accessed: 2025-08-25.
- [76] Shen Noether. Ring signature confidential transactions for monero. Cryptology ePrint Archive, Paper 2015/1098, 2015. <https://eprint.iacr.org/2015/1098>.
- [77] Oblivious message retrieval implementation. <https://github.com/ZeyuThomasLiu/ObliviousMessageRetrieval>, December 2021.
- [78] PALISADE lattice cryptography library (release 11.2). <https://palisade-crypto.org/>, June 2021.
- [79] Kabir Peshawaria, Zeyu Liu, Ben Fisch, and Eran Tromer. Laminate: Succinct SIMD-friendly verifiable FHE. Cryptology ePrint Archive, Report 2025/2285, 2025.
- [80] Sihang Pu, Sri AravindaKrishnan Thyagarajan, Nico Döttling, and Lucjan Hanzlik. Post quantum fuzzy stealth signatures and applications. *CCS 23*, 2023.

- [81] R L Rivest, L Adleman, and M L Dertouzos. On data banks and privacy homomorphisms. *Foundations of Secure Computation*, pages 169–179, 1978.
- [82] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the Association for Computing Machinery*, 21(2):120–126, February 1978.
- [83] Ron Rothblum. Homomorphic encryption: From private-key to public-key. In Yuval Ishai, editor, *TCC 2011*, volume 6597 of *LNCS*, pages 219–234. Springer, Berlin, Heidelberg, March 2011.
- [84] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [85] István András Seres, Balázs Pejó, and Péter Burcsi. The effect of false positives: Why fuzzy message detection leads to fuzzy privacy guarantees? In Ittay Eyal and Juan Garay, editors, *Financial Cryptography and Data Security*, pages 123–148, Cham, 2022. Springer International Publishing.
- [86] Muhammad Shahid. Zcash overview v3, 2024. Accessed: 2025-08-25.
- [87] Sandra Siby, Marc Juárez, Claudia Díaz, Narseo Vallina-Rodriguez, and Carmela Troncoso. Encrypted DNS $\implies$ privacy? A traffic analysis perspective. In *NDSS 2020*. The Internet Society, February 2020.
- [88] Jaspal Singh, Yu Wei, and Vassilis Zikas. Information-theoretic multi-server private information retrieval with client preprocessing. In Elette Boyle and Mohammad Mahmood, editors, *TCC 2024, Part IV*, volume 15367 of *LNCS*, pages 423–450. Springer, Cham, December 2024.
- [89] J. Szefer. Survey of microarchitectural side and covert channels, attacks, and defenses. *Journal of Hardware and Systems Security*, 3:219–234, September 2019.
- [90] The OpenSSL Project. Openssl: The open source toolkit for ssl/tls. <https://www.openssl.org/>, 2025. August, 2025.
- [91] David Isaac Wolinsky, Henry Corrigan-Gibbs, Bryan Ford, and Aaron Johnson. Dissent in numbers: Making strong anonymity scale. In *OSDI 12*, pages 179–182. USENIX, October 2012.
- [92] Kevin Yeo. Lower bounds for (batch) PIR with private preprocessing. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 518–550. Springer, Cham, April 2023.
- [93] Wenhao Zhang, Hanlin Liu, Kang Yang, Wen jie Lu, Yu Yu, Xiao Wang, and Chenkai Weng. BitGC made (more) efficient. Eurocrypt 2026, 2026.

- [94] Xinxuan Zhang, Ruida Wang, Zeyu Liu, Binwu Xiang, Yi Deng, Ben Fisch, and Xianhui Lu. Phalanx: An FHE-friendly SNARK for verifiable computation on encrypted data. In *ACM CCS 2025*, pages 4035–4048. ACM Press, November 2025.
- [95] Mingxun Zhou, Andrew Park, Wenting Zheng, and Elaine Shi. Piano: Extremely simple, single-server PIR with sublinear server computation. In *2024 IEEE Symposium on Security and Privacy*, pages 4296–4314. IEEE Computer Society Press, May 2024.

A Additional OMR Definition

Compactness. [58] additionally defines the compactness of OMR. In summary, it says that the digest size should essentially be at most polylogarithmic in N (and may be polynomial in $\bar{k}$ where $\bar{k} \leq N$ is the expected number of pertinent messages)

- (Compactness) An OMR scheme is compact if for $\lambda > 0, 0 < \epsilon_p, \epsilon_n < 1$, let $\text{pp} \leftarrow \text{GenParam}(1^\lambda, \epsilon_p, \epsilon_n)$, $(\text{sk}_{\text{clue}}, \text{pk}_{\text{clue}}) \leftarrow \text{OMR.GenClueKey}(\text{pp})$, $(\text{sk}_{\text{det}}, \text{pk}_{\text{det}}) \leftarrow \text{OMR.GenDetKey}(\text{pp}, \text{sk}_{\text{clue}})$, for any board $\text{BB} = \{(x_1, c_1), \dots, (x_N, c_N)\}$, letting $M \leftarrow \text{Retrieve}(\text{BB}, \text{pk}_{\text{det}}, \bar{k})$, it always holds that:

$$|M| = \text{poly}(\lambda, \log N) \cdot \log \epsilon_p^{-1} \cdot \tilde{O}(\bar{k} + \epsilon_p N) .$$

B BFV Additive Homomorphic Encryption

Plaintext encoding. In practice, instead of having a polynomial in $\mathcal{R}_t = \mathbb{Z}_t[X]/(X^D + 1)$ directly as input, applications usually hold a vector of messages $\vec{m} = (m_1, \dots, m_D) \in \mathbb{Z}_t^D$. Thus, to encrypt such input messages, BFV first encodes the messages into another polynomial $y(X) = \sum_{i \in [D]} y_i X^i$ such that $m_j = y(\zeta_j)$, $\zeta_j := \zeta^{3^j} \bmod t$, and ζ is the $2D$ -th primitive root of unity of t . Such encoding can be done using the Inverse Number Theoretic Transform (INTT), which is a linear transformation represented as matrix multiplication. We say that a BFV ciphertext has D slots, each of which is a $\mathbb{Z}_t$ element.

It is easy to see that to encode $\vec{m} := (x, x, \dots, x)$ of size D for any $x \in \mathbb{Z}_t$, the polynomial is $y(X) = x$, since y evaluated at any point is x .

During operations, we refer to these encoded $\mathbb{Z}_t$ vectors as “slots” and the coefficients of $y(X)$ encoding this vector as “coefficients”.

SlotToCoeff. One operation is called SlotToCoeff [25, 61, 62]. Essentially, this procedure *homomorphically* move all the $\mathbb{Z}_t$ elements inside the slots into coefficients: i.e., given a ciphertext encrypting $y(X)$ encoding $\vec{m}$ as above, this procedure terms it into $m(X) := \sum_{i \in [D]} \vec{m}[i] X^i$. This was first proposed for CKKS [25] and then officially adapted to BGV/BFV in [61]. Essentially, it is a linear transformation that can be represented as a matrix multiplication: i.e., given a ciphertext ct encrypting $y(X)$, there is some matrix $U \in \mathbb{Z}_t^{D \times D}$ (defined below) such that we can compute $\text{ct}' = M \times \text{ct}$ where M , and $\text{BFV.Dec}(\text{ct}') = m(X)$.

$$U := \begin{pmatrix} 1 & \zeta_0 & \zeta_0^2 & \dots & \zeta_0^{D-1} \\ 1 & \zeta_1 & \zeta_1^2 & \dots & \zeta_1^{D-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \zeta_{\frac{D}{2}-1} & \zeta_{\frac{D}{2}-1}^2 & \dots & \zeta_{\frac{D}{2}-1}^{D-1} \\ 1 & \bar{\zeta}_0 & \bar{\zeta}_0^2 & \dots & \bar{\zeta}_0^{D-1} \\ 1 & \bar{\zeta}_1 & \bar{\zeta}_1^2 & \dots & \bar{\zeta}_1^{D-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \bar{\zeta}_{\frac{D}{2}-1} & \bar{\zeta}_{\frac{D}{2}-1}^2 & \dots & \bar{\zeta}_{\frac{D}{2}-1}^{D-1} \end{pmatrix} \in \mathbb{Z}_t^{D \times D}$$

C Additively Homomorphic Secret Key Encryption Implies PIR

For simplicity, we define this primitive in $\mathbb{Z}_2$ to be the message space, but it trivially extends to other message space that allows addition.

Definition C.1 (Additively Homomorphic Symmetric Key Encryption). An Additively Homomorphic Symmetric Key Encryption (AHSKE) scheme consists of four PPT algorithms (GenParam , KeyGen , Enc , Dec) defined as follows:

- $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$: takes a security parameter λ ; outputs a public parameter pp .
- $\text{sk} \leftarrow \text{KeyGen}(\text{pp})$: takes the public parameter pp ; outputs a secret key sk .
- $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{sk}, m)$: takes the public parameter pp and the secret key sk , generates the ciphertext c for the message m in message space $\mathbb{Z}_2$; outputs ct .
- $m \leftarrow \text{Dec}(\text{pp}, \text{sk}, \text{ct})$: takes the public parameter pp , decrypts the ciphertext ct into a plaintext message m based on the secret key sk ; outputs the message m .

The scheme must satisfy the following properties:

- (Correctness) For any $\lambda > 0$, let $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$, let $\text{sk} \leftarrow \text{KeyGen}(\text{pp})$ and for any message m , let $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{sk}, m)$ and $m' \leftarrow \text{Dec}(\text{pp}, \text{sk}, \text{ct})$. It holds that if $\text{pp} \neq \perp$, $\Pr[m = m'] \geq 1 - \text{negl}(\lambda)$.
- (IND-CPA Security) For any PPT algorithm $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$: let $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$, $\text{sk} \leftarrow \text{KeyGen}(\text{pp})$; $\mathcal{A}_1$ chooses two messages and remembers state: $(m_0, m_1, \text{st}) \leftarrow \mathcal{A}_1^{\text{Enc}_{\text{sk}}}(\text{pp}, \text{pk})$; sample $b \leftarrow_{\$} \{0,1\}$ and $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{sk}, m_b)$; let $b' \leftarrow \mathcal{A}_2^{\text{Enc}_{\text{sk}}}(\text{st}, \text{ct})$; it holds that: $\Pr[b = b'] \leq 1/2 + \text{negl}(\lambda)$. Enc_{sk} means oracle access to the encryption procedure with secret key sk .
- (Additive Homomorphism) For any $\lambda > 0$, let $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$, let $\text{sk} \leftarrow \text{KeyGen}(\text{pp})$ and for any message $m_1, m_2 \in \mathbb{Z}_2$, let $\text{ct}_i \leftarrow \text{Enc}(\text{pp}, \text{sk}, m_i)$ for $i \in [1,2]$, and $m' \leftarrow \text{Dec}(\text{pp}, \text{sk}, \text{ct}_1 + \text{ct}_2)$.¹⁷ It holds that if $\text{pp} \neq \perp$, $\Pr[m = m_1 + m_2 \in \mathbb{Z}_2] \geq 1 - \text{negl}(\lambda)$.

AHSKE implies PIR. It is folklore that AHSKE implies PIR. However, to the best of our knowledge, no explicit construction/proof has been presented in the literature. We therefore show a simple lemma below.

Lemma C.2. *If there exists an additively homomorphic symmetric key encryption AHSKE, there exists a one-round PIR construction with communication cost $O(\sqrt{N})$ for a database $\in \{0,1\}^N$.*

Proof. Given AHSKE, we construct the PIR scheme PIR as follows (since it has only one round, we drop the subscript):

- $\text{PIR.GenParam}(1^\lambda, N)$:
 - $\text{pp}_{\text{AHSKE}} \leftarrow \text{AHSKE.GenParam}(1^\lambda)$
 - Return $\text{pp}_{\text{PIR}} = \text{pp}_{\text{AHSKE}}$

¹⁷For simplicity, assume $+$ is defined over ciphertext space in the natural way.

- $\text{PIR.Query}(\text{pp}_{\text{PIR}}^C, i) :$
 - Arrange database into $\sqrt{N}$ (WLOG assuming it is an integer) by $\sqrt{N}$ size matrix DB' where $\text{DB}[j][k] = \text{DB}[i]$ where $j = \lfloor i/\sqrt{N} \rfloor, k = i \bmod \sqrt{N}$.
 - $\text{sk} \leftarrow \text{AHSKE.KeyGen}(\text{pp}_{\text{AHSKE}})$
 - $\text{ct}_k \leftarrow \text{AHSKE.Enc}(\text{pp}_{\text{AHSKE}}, \text{sk}, 1)$ and $\text{ct}_l \leftarrow \text{AHSKE.Enc}(\text{pp}_{\text{AHSKE}}, \text{sk}, 0)$ for $l \in [\sqrt{N}] \setminus \{k\}$
 - Return $\text{qry} := (\text{ct}_l)_{l \in [\sqrt{N}]}, \text{st} := (\text{sk}, j)$
- $\text{PIR.Ans}(\text{pp}_{\text{PIR}}, \text{qry}, \text{DB}) :$
 - Arrange database into $\sqrt{N}$ (WLOG assuming it is an integer) by $\sqrt{N}$ size matrix DB' where $\text{DB}[j][k] = \text{DB}[i]$ where $j = \lfloor i/\sqrt{N} \rfloor, k = i \bmod \sqrt{N}$ for all $i \in [N]$
 - Return $\text{dig} \leftarrow \text{DB}' \times \text{qry}$
- $\text{PIR.Decode}(\text{pp}_{\text{PIR}}, \text{ans}, \text{st}) :$
 - Return $\text{res} \leftarrow \text{AHSKE.Dec}(\text{pp}_{\text{AHSKE}}, \text{sk}, \text{dig}[j])$

This scheme is a PIR construction in a straightforward way: correctness is guaranteed by the correctness of AHSKE and privacy is guaranteed by the IND-CPA security of AHSKE. Efficiency is also simple: the communication cost is simply $\sqrt{N}$ ciphertexts. And this scheme trivially extends to $\sqrt{N}$ not being an integer (rounding up and padding the database accordingly). $\square$

Remark C.3. This proof, together with Theorem 5.2, then shows that if PIR implies OMR, then AHSKE implies key-private PKE. This, to our knowledge, is still an open problem. There is a black-box construction of PKE from AHSKE [83]. However, it seems hard to obtain a black-box construction of *key-private* PKE from AHSKE, because homomorphism with respect to individual keys does not appear to help obtain indistinguishability between different keys.

For example, the PKE construction of [83] fails to obtain key-privacy for the following AHSKE. Suppose there exists an AHSKE Π , and consider the AHSKE Π' defined as follows, simply augmenting everything with a tag t that identifies the key:

- $\Pi'.\text{KeyGen}(\text{pp})$: $\text{sk}' = (\Pi.\text{KeyGen}(\text{pp}), t \leftarrow_{\$} \{0,1\}^\lambda)$
- $\Pi'.\text{Enc}(\text{pp}, (\text{sk}, t), m)$: $\text{ct}' = (\Pi.\text{Enc}(\text{pp}, \text{sk}, m), t)$.
- Addition: $(\text{ct}_1, t_1) +_{\Pi'} (\text{ct}_2, t_2)$: defined as $(\text{ct}_1 +_{\Pi} \text{ct}_2, t_1)$
- $\Pi'.\text{Dec}(\text{pp}, (\text{sk}, r), (\text{ct}, \tilde{r}))$: $\Pi.\text{Dec}(\text{pp}, \text{sk}, \text{ct})$ if $r = \tilde{r}$, otherwise $\perp$

This Π' is also an AHSKE, but the resulting PKE via [83] is not key-private, since its PKE ciphertexts consist of Π' ciphertexts all under the same key, which in the case of Π' preserves the t component, and thus uniquely identifies the key used. More generally, it is hard to imagine how a black-box construction applied to Π' would sanitize away the t component of Π' ciphertexts.

Note that this question is independent of the main contribution of this work, which is to build a practically efficient OMR construction, concretely matching the lower bound implied by the reduction from PIR.

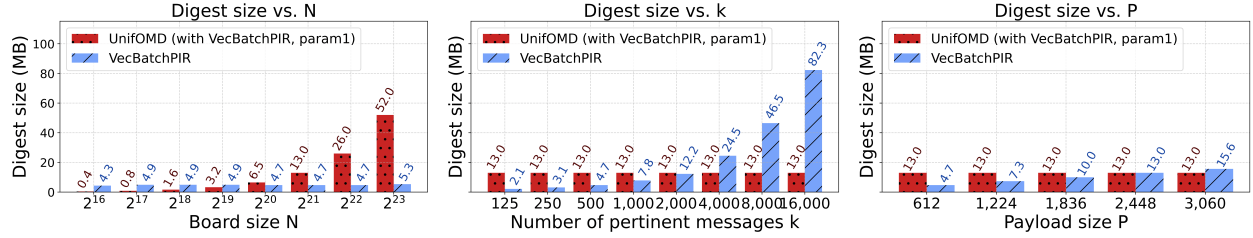


Figure 11: Digest size breakdown between detection (via UnifOMD, Param1) and retrieval (via VecBatchPIR [73]). Unless explicitly varied on the x-axis, we fix the parameters to $N = 2^{21}$, $P = 612$ bytes, and $k = 500$. From left to right, the plots are linear-log scaled, linear-log scaled, and linear-linear scaled, respectively.

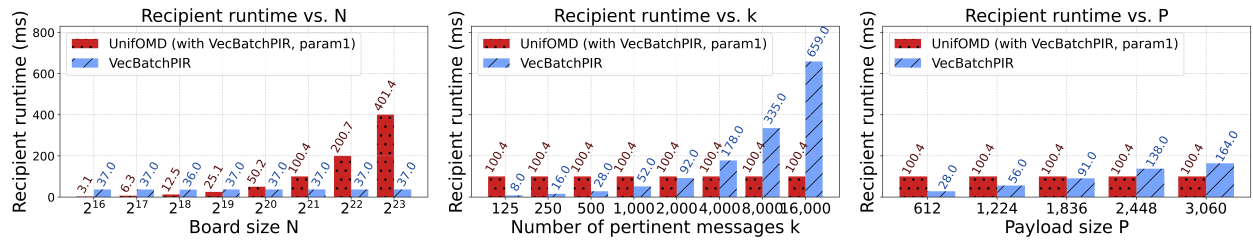


Figure 12: Recipient runtime breakdown between detection (via UnifOMD, Param1) and retrieval (via VecBatchPIR [73]). Unless explicitly varied on the x-axis, we fix the parameters to $N = 2^{21}$, $P = 612$ bytes, and $k = 500$. From left to right, the plots are linear-log scaled, linear-log scaled, and linear-linear scaled, respectively.

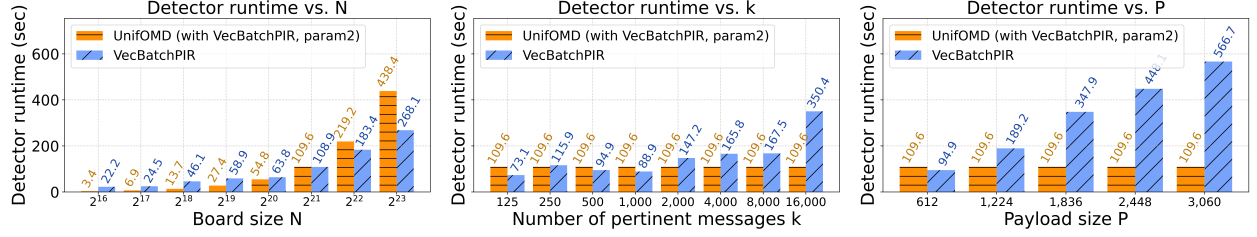


Figure 13: Detector runtime breakdown between detection (via UnifOMD, Param2) and retrieval (via VecBatchPIR [73]). Unless explicitly varied on the x-axis, we fix the parameters to $N = 2^{21}$, $P = 612$ bytes, and $k = 500$. From left to right, the plots are linear-log scaled, linear-log scaled, and linear-linear scaled, respectively.

D Additional Discussion on Evaluation

D.1 Baselines

We chose ECDH plus SKE in Section 7 as our baseline since, to our knowledge, this is the most efficient way to instantiate the baseline, and is used in Zcash [13, 86]. The mainstream alternatives include RSA [82] and Crystals-Kyber [84]. However, concretely:

- RSA has a ciphertext size of 256 bytes and a decryption time of 0.63ms [90].
- Crystals-Kyber has ciphertext size of 768 bytes and key decapsulating time of 0.06ms [84].
- ECDH has “ciphertext size” of 32 bytes and key decapsulating time of 0.05ms [90].

It is obvious that ECDH is the best choice in terms of concrete efficiency.

D.2 Additional Benchmarks

D.2.1 Parameter Set 1 Efficiency Breakdown

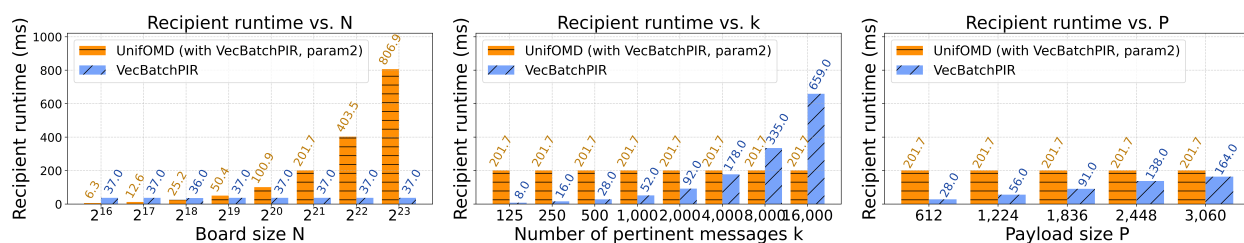
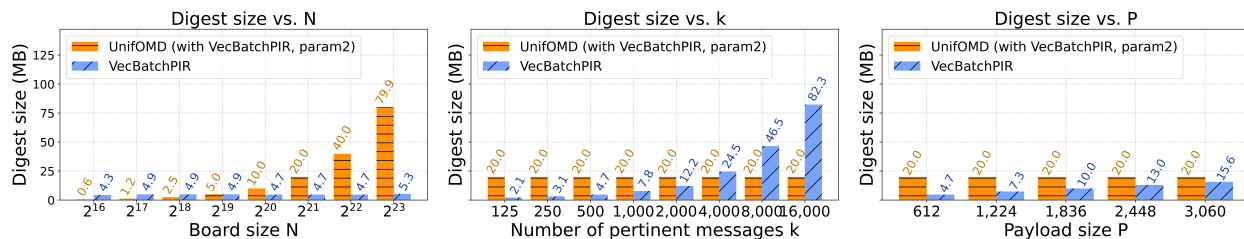
Figs. 11 and 12 show the digest size and recipient runtime breakdown for UnifOMR between detection via UnifOMD and retrieval via VecBatchPIR [73] with parameter set 1. As N grows larger, UnifOMD dominates while as k, P grows, VecBatchPIR dominates.

D.2.2 Parameter Set 2 Efficiency Breakdown

Figs. 13 to 15 show the detector runtime, digest size, and recipient runtime breakdown for UnifOMR between detection via UnifOMD and retrieval via VecBatchPIR [73], with parameter set 2. A similar conclusion can be drawn as for parameter set 1, except that UnifOMD efficiency is roughly $2\times$ worse in each metric.

D.3 Alternative Batch PIR

Instead of using VecBatchPIR, we could use other batch PIR constructions. Two other state-of-the-art batch PIR constructions are PIRANA [56] and OCC [15]. PIRANA has better computation



compared to **VecBatchPIR** but worse communication, and **OCC** has worse computation but better communication (for large payloads). However, unfortunately, the implementation of **OCC** is not publicly available, and for some parameters we are interested in, **PIRANA** has errors in their implementation. Thus, we could only use **VecBatchPIR**. Nevertheless, we would like to note that for **PIRANA**, the computation cost is less but still comparable to **VecBatchPIR**. Thus, changing from **VecBatchPIR** to **PIRANA** would not affect our claims.

D.4 Comparison to **InstantOMR** [54]

A very recent concurrent work **InstantOMR** [54] introduces another single-server FHE-based OMR construction. This section presents a more extensive comparison with **InstantOMR**.

Server runtime. For a large number of messages, such as $N \geq 2^{16}$ (and a single-core detector), **InstantOMR** has a worse detector runtime ($> 10\times$ worse than **SophOMR** and thereby $> 250\times$ worse than our **UnifOMR**). However, for a single message, **InstantOMR** takes < 0.1 second. In general, **InstantOMR** takes ~ 0.1 second per message for N messages, while for **SophOMR**, the runtime is almost identical to having 2^{16} messages even when $N \ll 2^{16}$ (since the underlying BFV requires **SophOMR** to batch 2^{16} messages to achieve its best detector runtime). Thus, for a small batch of messages, **InstantOMR** can outperform **SophOMR**. However, since **InstantOMR** is $> 250\times$ slower than **UnifOMR**, to outperform **UnifOMR**, $N \lesssim 10$ is needed (since we only need to batch 2048 messages to maximize our efficiency for the OMD component of our **UnifOMR**), which may be too small for real-world applications.

Communication costs. **InstantOMR** shares a similar digest size as **SophOMR** (about $4\times$ worse than **SophOMR** but also asymptotically compact). Thus, the communication comparison is similar to our comparison with **SophOMR**, so we skip the details.

Latency. As mentioned in Section 7, **InstantOMR** [54] focuses on the “latency” of OMR. Essentially, latency refers to the time between when the recipient submits the retrieval request and when the recipient receives the digest. In regular settings, the latency is essentially the detector runtime for **InstantOMR** and **SophOMR**. **InstantOMR** is mainly concerned with the case when the number of messages is small, such as a single message (which is more reasonable in the application with streaming updates, discussed in more detail below).

For **UnifOMR**, latency is slightly more complicated as we have two rounds. Thus, to measure latency, we first use the traffic control tool ‘tc’ to emulate network latency (30 ms) and enforce a bandwidth limit of 1 gigabit/s (emulating the wide area network, WAN) to test a two-party two-round communication time (with given size of digests). This time is used together with the runtime (including client and server runtime) of the underlying batchPIR construction. Under such tests, we show that for the parameters tested in [54], $N = 2^{16}$, $k = 50$, our latency is < 8 seconds, orders of magnitude faster than **InstantOMR**. Furthermore, even under the extreme case, where there is only a single message, our latency is < 2 seconds. While this is slower than **InstantOMR**, which is < 0.1 second, we believe such latency is sufficient for applications (and $> 25\times$ smaller than **SophOMR**).

Note that this test also confirms our end-to-end runtime is still dominated by the detector runtime, *not* communication.

Streaming updates. The main application scenario proposed by **InstantOMR** is “streaming updates”. In this setting, the recipient uploads its detection key to the server in advance. When new messages arrive, the detector uses the uploaded key to accumulate the digest. In this case, when

becoming online, the recipient may be able to obtain the digest immediately: for `InstantOMR`, if there are n newly arriving messages the moment that the recipient is online, it takes $\sim 0.1 \cdot c$ seconds for the recipient to receive the digest (regardless of N , which includes the messages accumulated offline, being already processed by the detector). In this setting, the latency is between the time the recipient becomes online and the time the recipient receives its digest.

In `UnifOMR`, this runtime is mostly the PIR runtime, which can be tens of seconds for millions of messages, which is a noticeable difference compared to `InstantOMR`. Thus, in the application where streaming updates are necessary and the server is highly powerful (which can afford spending ~ 0.1 second per message per core), `InstantOMR` can be preferred over `UnifOMR`. However, in other cases (either streaming updates are not needed or the server is less powerful), `UnifOMR` is preferred.

Multi-threading. Another advantage of `InstantOMR` over `SophOMR` is its inherent capability of being parallelized. For c CPU cores, the detector runtime of `InstantOMR` is the same for 1 message and c messages with multi-threading. This means that for a powerful server (e.g., > 250 cores), the detector runtime of `InstantOMR` can be as fast as single-threaded `UnifOMR`.

However, we would like to note that `UnifOMR` can also be potentially multi-threaded: as long as the underlying batchPIR scheme can be multi-threaded. Our OMD component is almost perfectly multi-threaded (i.e., achieving the same capability as `InstantOMR`), since our BFV multiplication is all in NTT form. Thus, as long as the batchPIR we rely on benefits from multi-threading, `UnifOMR` can be greatly parallelized as well. We leave this as a future direction to explore.

E Open Questions

While our scheme establishes the theoretical and concrete foundations for OMR, there are still a lot of interesting open questions.

Circumventing/Tightening the lower bound. One interesting direction is to build a non-detection-key-unlinkable OMR construction. This immediately avoids our lower bound. However, it is not quite clear how this helps efficiency. For example, the general scheme via FHE introduced in [58] indeed does not have detection-key-unlinkability. However, it is still asymptotically as slow as the prior works (and standard PIR) and concretely even a lot worse. Thus, it is unclear why this would help. An alternative question is whether we can prove a lower bound for OMR without strong-detection-key-unlinkability.

What compact OMR implies. Right now, we show that non-compact OMR implies key-private PKE with wrong-key-decryption. However, as shown by [58], OMR is implied by FHE. There is a certain gap between key-private PKE and FHE. Thus, it would be interesting to see if the compactness of OMR can help show that OMR implies a stronger primitive.

Proving PIR does not imply OMR. While our proof (Theorem 5.2) provides strong evidence that PIR does not imply OMR, it is not formally proven. It would be interesting to see if this could be proven or disproved.

The use of stateful PIR. In `UnifOMR`, we use the most standard version of PIR: not client-preprocessing [30, 95, 34] nor global preprocessing [55, 43, 53, 69]. In client-preprocessing PIR schemes, each client holds some hint of the database, which can only be obtained by streaming the whole database offline [95] or using very heavy FHE [34]. However, neither is suitable for OMR. On the other hand, global silent preprocessing [53, 69] can be more relevant. However, these schemes now have relatively large communication costs. Directly making it batchPIR can easily cause close

to a hundred megabytes of communication for the parameters we are interested in, as in Section 7. Thus, a study on making stateful PIR concretely more suitable for OMR is another interesting direction.

Single-server OMR with a different assumption. To our knowledge, all existing compact single-server OMR constructions rely on lattice-based assumptions. An interesting open question is whether one can construct a (compact) OMR scheme from alternative assumptions (e.g., leveraging leveled homomorphic encryption under different hardness assumptions [29]).

Integrity of single-server OMR. For single-server OMR, a long-standing open problem is achieving integrity. Current constructions guarantee correctness only under a semi-honest server assumption, while privacy is proven against malicious servers. Bridging this gap remains an important challenge. A natural approach to achieving integrity is to equip the server with a proof of correct evaluation. One promising direction is to generate such proofs homomorphically (e.g., [36, 5, 35, 94, 79]).

Interestingly, the existing OMR standard privacy notion is “CPA-like,” in the sense that the adversary does not learn any information from recipients after retrieval. While this assumption can be reasonable, particularly in the presence of key unlinkability, it is natural to ask whether one can strengthen security to a “CCA-like” notion. Achieving such a guarantee appears to be non-trivial, even if integrity is assumed. In particular, lattice-based constructions may be vulnerable to so-called IND-CPA-D attacks [52, 24, 23, 14, 63], where even the decryption of honestly generated ciphertexts can leak information about the secret key. As a result, obtaining full “CCA-like” security for single-server OMR likely requires additional care and new techniques.

Multi-server OMR. Existing multi-server OMR constructions rely on communicating servers [66, 47, 27]. A natural direction for future work is to explore constructions based on non-communicating servers, similar to the setting of multi-server PIR (e.g., [10, 30, 88, 39, 50, 44]). One possible approach is to build OMR using homomorphic secret sharing [19, 20, 6, 93, 48], which is widely open since HSS mainly focuses on evaluating branching programs. Another direction is to relax the environment assumptions: rather than assuming semi-honest servers for privacy¹⁸, one could instead consider rational adversaries (e.g., [40, 41, 42]).

¹⁸[27] considers malicious servers, but at a non-negligible performance cost.